SermonFlow Backend: The Complete Guide

Oct 8, 2026 · @Bojuto

1. The big picture

SermonFlow takes a church's sermon (audio, video or a document), turns it into a transcript and study notes, then generates social posts, reels, devotionals and other content in the church's brand, and publishes them. The backend is one NestJS codebase (about 44,000 hand-written lines in 349 files) that runs as **two processes**:

Process	Entry file	What it does
API	`src/main.ts`	Serves HTTP requests from the web app. Never does slow work itself; it puts jobs on a queue.
Worker	`src/worker.ts`	No HTTP. Pulls jobs from BullMQ queues (transcription, AI analysis, embeddings, content generation, rendering, publishing, email, cleanup) and runs them.

Both processes load the same `AppModule` and the same use cases. They are deployed and scaled separately, so a burst of transcription jobs never slows down the website.

How to read this guide

This guide goes **file by file**. For each file it says what the file is for, points at the important lines, and explains *why* they are written that way. Line-by-line commentary is kept for the files that matter most (entry points, auth, the shared plumbing). Explaining all 44,000 lines one at a time would make the guide longer than the code, and many lines are simple imports or type declarations. Section 11 has recipes for making changes.

The four layers, in plain words

Every business module (auth, sermons, content, social, and so on) has the same four folders. Think of a restaurant:

Folder	Restaurant role	Rule
`domain/`	The recipes	Pure TypeScript business rules. Imports nothing from NestJS, Prisma, Redis or any vendor SDK.

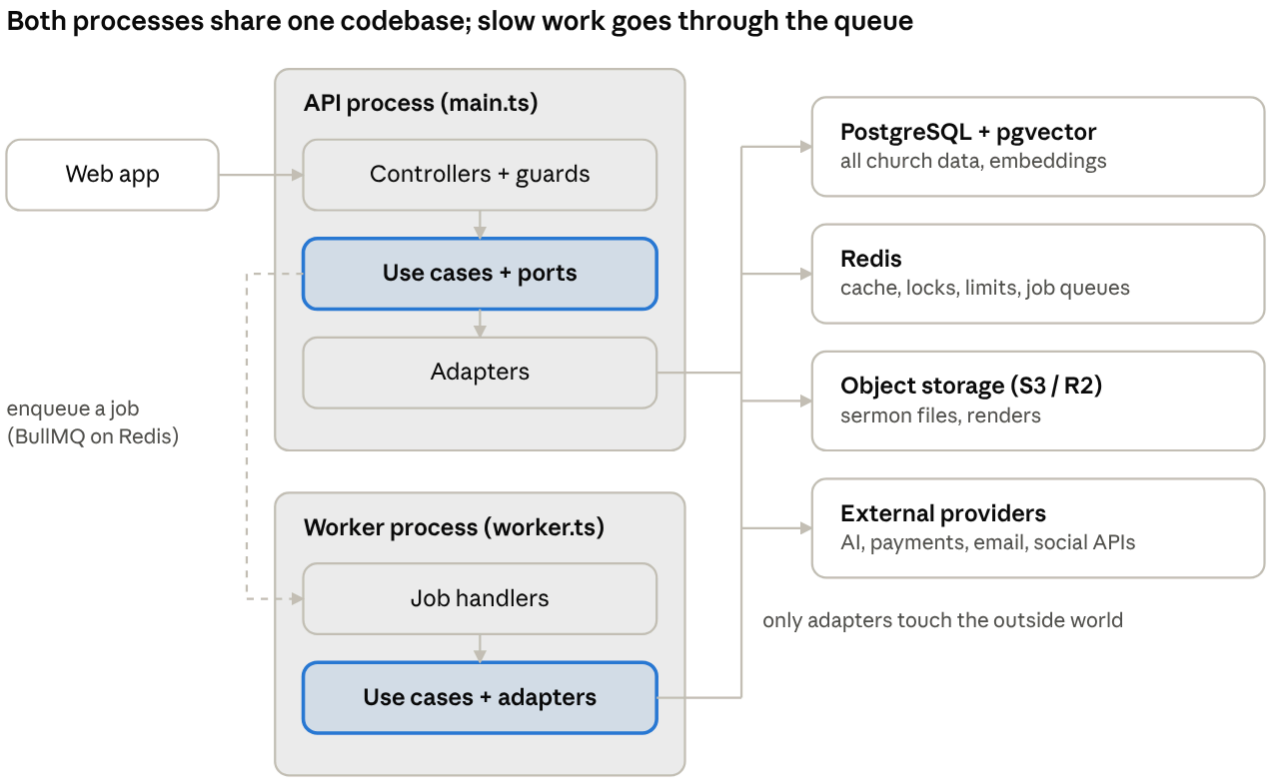
Folder	Restaurant role	Rule
`application/`	The head chef	Use cases, one class per action ("log in", "generate content"). Talks to the outside world only through **ports**: interfaces such as `UserRepositoryPort`.
`infrastructure/`	The suppliers	**Adapters** that implement ports: Prisma repositories, the S3 client, the AI client, the scrypt hasher.
`presentation/`	The waiters	Entry points: HTTP controllers in `presentation/http` and queue job handlers in `presentation/queue`. They validate input, call exactly one use case and shape the reply.

Two extra places tie it together:

- `src/composition/` is the **only** place where a port is connected to its adapter. It is like a switchboard. To swap OpenAI for another AI vendor, you change one line here plus a new adapter.
- `src/shared/` holds the plumbing every module uses: config, logging, Redis, the queue, the cache, retries and circuit breakers, storage, email and AI clients.

Life of one request

1. **Global middleware and guards** run first: security headers, CORS, CSRF origin check, the rate limiter (Redis-backed), and the session guard. Every route needs a logged-in session unless it is marked `@Public()`.
2. **The controller** validates the body with a Zod schema (bad input becomes a 400) and gets `churchId` from the session with `@CurrentChurchId()`, never from the request body.
3. **The use case** applies the business rules, calls ports and throws `DomainError` when something is wrong.
4. **Adapters** do the I/O. Every repository query filters by `churchId`, so one church can never see another church's data.
5. **Slow work** (for example transcribing an hour of audio) is not done in the request. The use case enqueues a job and returns straight away. The worker picks the job up, and the web app polls or refreshes to see progress.
6. **Errors** go through one exception filter that turns `DomainError` into a clean JSON envelope (`statusCode, code, message, errors, requestId`). Unknown errors become an opaque 500 and are sent to Sentry.



architecture · API, worker and the shared stores

The API answers the web app and hands slow work to the worker through Redis; both reach the database, storage and providers only through adapters.

The modules at a glance

Module	What a church gets from it
`auth`	Sign up, log in, sessions, password reset
`church`	The church profile (name, denomination, onboarding)
`uploads`	Safe direct-to-storage file uploads
`sermons`	Transcripts, sermon intelligence, scripture detection, embeddings, the processing pipeline
`content`	AI-generated content, versions, the content calendar
`library`	Search and "ask your sermons" (semantic search)
`brand`	Brand Studio: colours, fonts, logos, branded media

Module	What a church gets from it
`reels`	Picking reel-worthy moments, hooks, branded 9:16 video renders
`social`	Connecting social accounts and publishing directly
`publishing` / `integrations`	Publishing through connected integrations; encrypted credentials and OAuth
`billing` / `usage`	Plans, quotas, Paystack payments, usage counting
`analytics`	Usage and performance numbers
`health` / `system`	Liveness and readiness checks, metrics, email and maintenance jobs

2. Packages: what each one does and why

Every dependency in `package.json` falls into one of the groups below. Runtime packages ship in the Docker image; dev packages are only for building, linting and testing. The project is ESM (`"type": "module"`), so relative imports end in `.js` .

Framework (runtime)

Package	What it does	Why we use it
`@nestjs/core` , `@nestjs/common`	The NestJS framework: modules, dependency injection, controllers, guards, filters	Gives the app a clear structure and DI, which the hexagonal layout relies on to plug adapters into ports
`@nestjs/platform-express` + `express` (v5)	The HTTP server underneath Nest	Express is the most widely supported Nest platform; v5 handles async errors properly
`@nestjs/config`	Loads environment variables	Paired with a Zod schema so a missing or wrong variable stops the app at boot

Package	What it does	Why we use it
`@nestjs/throttler`	Rate limiting	Global request limits, tighter limits on login and AI routes; storage is in Redis so the limit holds across replicas
`@nestjs/locks`	Distributed locks for scheduled jobs	`@Cron` fires on every replica; a Redis lock makes sure only one runs each tick
`@nestjs/swagger`	Generates API docs	Handy for the frontend team; disabled or protected in production
`reflect-metadata`, `rxjs`	Required internals of NestJS	Nest's decorators and interceptors depend on them
`cookie-parser`	Reads cookies from requests	The session token lives in an `HttpOnly` cookie

Data and queues (runtime)

Package	What it does	Why we use it
`@prisma/client`, `@prisma/adapter-pg`, `pg`	Prisma ORM, using the `pg` Postgres driver	Type-safe queries and parameterised SQL (no SQL injection); the pg adapter lets us set pool size and statement timeout explicitly
`ioredis`	Redis client	Used for the cache, session cache, rate limits, locks, idempotency records and BullMQ
`bullmq`	Job queue on top of Redis	Moves slow work (transcription, AI, rendering, publishing) to the worker with retries and backoff
`zod` (v4)	Schema validation	One tool for validating HTTP bodies, env vars and AI JSON output

Files, media and documents (runtime)

Package	What it does	Why we use it
`@aws-sdk/client-s3`, `@aws-sdk/s3-request-presigner`	Talks to S3-compatible storage (R2, S3, MinIO locally)	Files go straight from the browser to storage using presigned URLs, so large uploads never pass through the API
`sharp`	Fast image processing	Resizing and converting brand assets and thumbnails
`unpdf`	Extracts text from PDFs	Lets a church upload a sermon as a PDF instead of audio
`mammoth`	Extracts text from Word `.docx` files	Same as above, for Word documents

Logging, metrics and errors (runtime)

Package	What it does	Why we use it
`pino`, `pino-http`, `nestjs-pino`	Structured JSON logging	Fast logs with a request id on every line, as the rules require
`prom-client`	Prometheus metrics	Per-route latency, error rates and queue stats for dashboards and alerts
`@sentry/node`	Error tracking	5xx errors and crashes are reported with stack traces (request bodies and cookies are stripped). Loaded first via `src/instrument.ts`

There is no AI SDK in the list. The AI, transcription, email and payment providers are called over plain `fetch` from adapters in `src/shared/infrastructure`, each with an explicit timeout and a circuit breaker.

Development only

Package	What it does	Why we use it
`typescript` (v6)	The compiler	Type checking with `npm run typecheck`
`@nestjs/cli`, `@nestjs/schematics`	`nest build`, `nest start --watch`, generators	Standard Nest build and dev server
`@nestjs/mau`	Nest's deployment CLI	Optional tooling from the Nest team
`prisma`	Prisma CLI	Generates the client and runs migrations
`vitest`, `@vitest/coverage-v8`, `vite-tsconfig-paths`	Test runner and coverage	Much faster than Jest. It uses esbuild, which does not emit decorator metadata, which is why every injection uses explicit `@Inject(TOKEN)`
`@nestjs/testing`, `supertest`	Spin up the app and send fake HTTP requests	Used by `createTestApp` in the e2e tests

Package	What it does	Why we use it
`oxlint`, `oxlint-tsgolint`	Linter	A Rust linter that is far faster than ESLint, with type-aware rules
`prettier`	Code formatter	Consistent formatting
`pino-pretty`	Makes logs readable in the terminal	Local development only
`source-map-support`	Maps stack traces to `.ts` lines	Debugging help (production uses `--enable-source-maps` instead)
`@types/*`	Type definitions	Types for Node, Express, pg, cookie-parser and supertest

Other `package.json` entries

- **`overrides`** force patched versions of `mysql2` and `deepmerge-ts`. These are transitive dependencies (pulled in by other packages), so pinning them here fixes known security issues without waiting for upstream.
- **`allowScripts`** lets only Prisma run install scripts. Other packages' postinstall scripts are blocked, which protects against supply-chain attacks.
- **Key scripts:** `start:dev` / `start:dev:worker` (local development), `start:api` / `start:worker` (production, run with `--enable-source-maps` and Sentry preloaded), `db:migrate` (production migrations), `db:check` (makes sure hand-written indexes aren't dropped) and `verify` (lint, typecheck, all tests and build; run it before finishing any change).

3. Project setup: config, Docker, CI and scripts

These files never run inside a request. They decide how the code is compiled, tested, packaged into an image and shipped.

Local development flow: `docker compose up` starts Postgres (with pgvector), Redis and RustFS. Then `npm run storage:init` creates the bucket, `npm run db:migrate:dev` and `npm run db:seed` prepare the database, and you run `npm run start:dev` plus `npm run start:dev:worker` in two terminals. Copy `.env.example` to `.env` first.

Shipping flow: push or open a PR, then `ci.yml` runs (lint, typecheck, tests, migrations on a throwaway DB, `check-migrations.mjs`, build, audit, docker build). When main passes, `deploy.yml` builds the `runtime` and `migrate` images and runs `deploy.sh` in order:

migrate, then api, then worker. It then polls `/health/ready`.

Compiler and test config

tsconfig.json

- `module` / `moduleResolution: nodenext` means real ESM. That is why relative imports need `.js` on the end.
- `target ES2023` with `strict` on. `strictPropertyInitialization` is off because Nest fills class fields through DI.
- `experimentalDecorators` + `emitDecoratorMetadata` turn on Nest's decorators. `sourceMap` makes `--enable-source-maps` point stack traces at `.ts` lines.

tsconfig.build.json

- Used by `nest build`. It compiles only `src/` and leaves out tests and `*spec.ts`, so `dist/` holds only app code (`main.js`, `worker.js`, `seed.js`, `instrument.js`).

nest-cli.json

- Source root is `src`. `deleteOutDir: true` wipes `dist` before each build so stale compiled files never linger.

vitest.config.ts

- Unit tests are every `*.spec.ts`, with globals on (`describe` / `it` need no import). Vitest uses esbuild, which emits no decorator metadata. That is the reason for the house rule: always write `@Inject(TOKEN)`.

vitest.config.e2e.ts

- e2e tests are `*.e2e-spec.ts`. `fileParallelism: false` (L12) because each suite boots the whole app and hashes passwords; running them in parallel caused rare socket resets.

prisma.config.ts

- Prisma 7 CLI config: the schema path, migrations folder, seed command and `DATABASE_URL` (L10-19). It loads `.env` only if the file exists (local CLI use); deployed environments inject variables directly.

.env.example

- The documented list of every environment variable, with safe development defaults, grouped by prefix: `APP_*`, `DATABASE_*`, `REDIS_*`, `AUTH_*`, `STORAGE_*`, `AI_*`, `TRANSCRIPTION_*`, `WORKER_*`, `BILLING_*`, `EMAIL_*`, `PUBLISHING_*`, `SOCIAL_*`, `MEDIA_*`, `OBSERVABILITY_*`.
- The AI, transcription, billing and email providers default to fake or outbox versions, so the app runs locally with no API keys at all.
- `DATABASE_STATEMENT_TIMEOUT_MS` gives every query a time limit. `WORKER_SHUTDOWN_GRACE_MS` must not exceed the hosting platform's grace period. `APP_SHUTDOWN_DRAIN_MS` is the pre-stop delay.
- Secret values are left empty. In production they come from the platform's secret store, never from the image.

Docker and local services

Dockerfile

- **Multi-stage build** (L4-13). `deps` runs `npm ci --ignore-scripts` from the lockfile. `build` runs `prisma generate` and `npm run build`, then prunes dev dependencies.
- **`runtime` stage** (L15-40) sets `NODE_ENV=production` and installs ffmpeg (for media), fonts (without them branded text renders blank) and tini. It copies only `node_modules`, `dist`, `package.json` (Node needs it to see `"type": "module"`) and `prisma`. No source code and no `.env`.
- L30 `USER node`: the container does not run as root. L32-33 a Docker HEALTHCHECK hits `/health`.
- L34 `ENTRYPOINT tini --`: tini passes SIGTERM to Node and cleans up child processes (ffmpeg, Python).
- L40 exec-form CMD: `node --enable-source-maps --max-old-space-size=384 --import ./dist/instrument.js dist/main.js`. 384 MB is about 75% of a 512 MB container, so a memory leak produces a stack trace instead of a silent kill (exit 137). The worker overrides this with 1536 MB and `dist/worker.js`.
- L43-49 **`migrate` stage**: a full `npm ci` (it needs the Prisma CLI) that runs `prisma migrate deploy` as its own deploy step, never at app startup.

docker-compose.yml

- Local development only. Postgres 17 with pgvector on host port 5433, Redis 7 on 6380 (append-only persistence), RustFS S3-compatible storage on 9000/9001.
- Every port binds to `127.0.0.1`, so nothing is exposed on your network. `rustfs/rustfs:latest` is an unpinned tag.

docker/postgres/init.sql

- `CREATE EXTENSION IF NOT EXISTS vector;` turns on pgvector, which stores sermon embeddings for semantic search.

.dockerignore

- Keeps `node_modules`, `dist`, `.env` and `.env.*` (except `.env.example`), `.outbox`, `.git`, `src/generated` and logs out of the image, so secrets never get baked in.

CI/CD

.github/dependabot.yml

- Weekly small update PRs for npm, the Docker base image and GitHub Actions. It groups the `@nestjs/*`, Prisma and AWS SDK families so each family moves together.

.github/workflows/ci.yml

- Runs on every PR and every push to main. `concurrency` cancels older runs on the same branch, with a 20-minute timeout.
- Starts a throwaway pgvector Postgres, then runs: `npm ci`, `prisma generate`, lint, typecheck, unit tests, e2e tests (in-memory adapters), `prisma migrate deploy` on the empty DB, `check-migrations.mjs`, build, `npm audit --omit=dev --audit-level=high` and `docker build --target runtime`.
- Tests never touch that database. It exists only to prove the migrations apply.

.github/workflows/deploy.yml

- Runs after CI succeeds on main (deploys to staging), or by hand with a choice of staging or production. Production approval comes from GitHub environment protection.
- Job `image` builds and pushes two images to GHCR, tagged with the commit SHA: `runtime` and `<tag>-migrate`.
- Job `deploy` runs `deploy.sh migrate`, then `api`, then `worker`, and then polls `$HEALTH_URL/health/ready` up to 30 times, 10 s apart.

scripts/deploy.sh

- A **placeholder**. It describes the three roles (migrate once, api replicas, worker replicas) but **always exits 1** (L14-15) because no hosting platform has been chosen yet. The deploy workflow will fail until this is implemented.

scripts/check-migrations.mjs (`npm run db:check`)

- Part 1 (L22-47) diffs the migrated database against `schema.prisma`. Any difference is drift, except dropping the 7 hand-written trigram/HNSW indexes that Prisma cannot express.
- Part 2 (L49-68) scans every `migration.sql` for destructive SQL (`DROP TABLE/COLUMN` , `TRUNCATE` , `DELETE FROM` , column type changes) and blocks it. A reviewed expand/contract step can opt out with the comment `-- abyde:allow-destructive` .

scripts/storage-init.mjs

- Creates the private dev bucket (safe to re-run) and a CORS rule allowing PUT and GET from the web app, because browsers upload sermon files straight to storage.

scripts/smoke.mjs (`npm run smoke`)

- An end-to-end check against a running API and worker that uses only the public HTTP API. Without an audio file it generates a 90 s tone with ffmpeg.
- The flow: register, create a church, upload, create a sermon, poll until READY, read the transcript and intelligence, generate a week of content, edit, approve, schedule, search, ask, view the dashboard and usage, then Reels. With `SMOKE_FAKE_BILLING=1` it upgrades through the fake billing webhook and renders a real Reel.

scripts/speaker-tracking/ (Python)

- `track_speaker.py` is called when `MEDIA_SPEAKER_TRACKER=command` . It finds the preacher's face in each frame (MediaPipe + OpenCV) and prints JSON `{"samples": [{t, cx, confidence}]}` , so a Reel can follow the speaker instead of cropping the centre.
- It favours the face that scores best on detection confidence, size and closeness to the previous pick, so a front-row face or a cutaway does not steal the frame. Any failure falls back to a centre crop.

deploy/alerts/prometheus-rules.yml

- Alert rules for `/metrics` : 5xx rate above 2% (page someone), p95 latency above 2 s, more than 3 restarts in 30 min, heap above 90%, queue backlog above 100, failing jobs, the every-minute `dispatch-due` scheduler stalling (page), the AI provider failing above 20%, and estimated AI spend above $20 an hour.

4. Bootstrap: how the app starts and gets wired together

The API starts like this: `node --import dist/instrument.js dist/main.js`. `instrument.ts` loads Sentry first, then `main.ts` calls `AppModule.register()`. That validates the environment, the `composition/` modules plug adapters into ports, and `configureApp()` adds headers, CSRF, CORS, cookies and the `/api/v1` prefix before `app.listen()`.

Every request then goes through: security headers, CSRF and CORS, cookie-parser, the pino logger (assigns a request id), `ThrottlerGuard` (rate limit in Redis), the session guard, `HttpMetricsInterceptor`, `IdempotencyInterceptor`, then the controller, the use case, the ports and the adapters. Errors are shaped by `ErrorEnvelopeFilter`.

Entry points

src/main.ts, line by line

```
async function bootstrap() {
  crashOnFatalErrors();
  const app = await NestFactory.create<NestExpressApplication>(
    AppModule.register(),
    { bufferLogs: true, rawBody: true },
  );
  const config = app.get<AppConfig>(APP_CONFIG);
  configureApp(app, config);
  await app.listen(config.APP_PORT);
}
await bootstrap();
```

- `crashOnFatalErrors()`: if an unhandled error or rejected promise happens, log the stack and exit. A half-broken process is worse than a clean restart.
- `NestFactory.create(AppModule.register(), …)`: builds the whole dependency graph. **Environment validation runs here**, so a missing or bad env var stops the process before it takes any traffic.
- `bufferLogs: true`: hold early log lines until the pino logger is attached, so every line is structured JSON.
- `rawBody: true`: keep the exact bytes of each request body. Webhook signature checks (for example Paystack) must hash the raw bytes, not re-serialised JSON.
- `app.get(APP_CONFIG)`: read the already-validated config object.
- `configureApp(app, config)`: the HTTP pipeline, kept in a separate file so the e2e tests use exactly the same setup.
- `app.listen(config.APP_PORT)`: start accepting requests.

- `await bootstrap()`: top-level await works because this is an ES module.

src/worker.ts

- Starts the **worker** (21 lines). It uses the same modules and use cases as the API, but `createApplicationContext` (L14) creates no HTTP server.
- L15 adds `BullMqWorkerHost` as an extra provider. Only the worker consumes queues.
- L19 `enableShutdownHooks()`: on SIGTERM, workers and connection pools close cleanly and running jobs get their grace period.
- Why a separate process: heavy jobs (ffmpeg, AI) can be scaled and given more memory without touching the API.

src/bootstrap.ts: `configureApp()`

- L14 `API_PREFIX = 'api/v1'`.
- L24 `trust proxy` = `APP_TRUST_PROXY_HOPS`, a number and never `true`. Rate limits then see the real client IP, and a client cannot fake its own IP.
- L29-40 `app.useSecurityHeaders()` (Nest 12). The CSP is `default-src 'none'` because the API returns only JSON. The CSP is relaxed only when Swagger is on, so its UI can load.
- L43 `enableCsrfProtection({ trustedOrigins: APP_CORS_ORIGINS })`: browser requests that change state are rejected when they come from another site. Webhooks send no `Origin` header, so they pass.
- L44-52 body size: the global JSON limit stays at 100 KB. Only `POST /api/v1/sermons` (a pasted transcript) gets 2 MB. This follows the rule: never raise the global limit for one route.
- L53 `cookieParser()`, because sessions are cookies. L54-61 CORS with an explicit origin list and `credentials: true`. It exposes `Idempotent-Replayed`, `Retry-After` and `X-Request-Id` so the web app can read them.
- L62-64 prefixes every route with `/api/v1` except `health`, `health/ready` and `metrics`, which load balancers and Prometheus call at fixed paths.
- L67-77 Swagger at `/docs`, only when `config.swaggerEnabled` is true (off in production by default).

src/instrument.ts

- Preloaded with `node --import`, so Sentry starts **before** any other module. Automatic tracing of HTTP, Postgres and Redis only works when it loads first.
- It runs before ConfigModule exists, so it reads 3 env vars directly and clamps the trace sample rate to 0..1. Those vars are still validated at boot along with everything else.

src/app.module.ts: the root module

- `AppModule.register(options)` (L79) returns a dynamic module, so **tests can swap the infrastructure**. `options.infrastructure` replaces the production set (Core, Prisma, ExternalProviders) with in-memory versions from `test/support/`.
- L59-75 `BUSINESS_MODULES` lists every feature module. Add a new module here.
- L90-127 the pino logger:
  - The request id reuses an incoming `x-request-id` only if it matches `^[A-Za-z0-9-]{8,128}$`, which stops log injection. Otherwise it generates a random UUID and echoes it back in the response header.
  - It redacts the auth header, cookies, `set-cookie` and the Paystack signature.
  - It logs only id, method and path. No query strings and no bodies, so sermon text stays out of logs.
  - It skips `/health` and `/metrics`. Pretty logs appear only in development.
- L128-140 `ThrottlerModule`: a global limit with **shared Redis storage**, so the limit is not multiplied by the number of replicas.
- L143-149 global providers registered as `APP_FILTER`, `APP_GUARD` and `APP_INTERCEPTOR` (not with `app.useGlobal*`), so tests behave like production: `ErrorEnvelopeFilter`, `ThrottlerGuard`, `HttpMetricsInterceptor` and `IdempotencyInterceptor`. The global auth guard is registered inside the auth module.

src/seed.ts (`npm run db:seed`)

- Development only. It creates two fictional churches with one demo sermon each, and refuses to run when `APP_ENV` is `staging` or `production`.
- It is idempotent: a demo user whose email already exists is skipped.
- It goes through the **real use cases** (`RegisterUseCase`, `CreateChurchUseCase`, `CreateSermonUseCase`) instead of writing rows directly, so every business rule applies. A `finally` block always closes the app.

src/composition/: which adapter plugs into which port

A use case asks for a token, such as `@Inject(STORAGE)`. Nest finds the provider registered in one of these `@Global()` modules and hands over the concrete adapter. Business modules never pick adapters themselves. Tests replace all three modules with in-memory ones.

tokens.ts

- One line: the `THROTTLER_STORAGE` symbol, the DI token for the rate-limit store.

core-infrastructure.module.ts

- The shared infrastructure every module uses: Postgres (`PrismaService` , `PrismaUnitOfWork`), Redis (`RedisConnection` , `RedisCacheAdapter`), the BullMQ queue, the clock, crypto and metrics.
- L48 `LocksModule.forRoot({ isGlobal: true, ttl: '60s' })` + `RedisLockStore` : a scheduled job takes a lock in Redis, so only one replica runs each tick. `FencingRegistry` rejects writes from a holder that has lost its lock.
- L52 `ConnectionBudgetCheck` : checks that pool size × expected processes fits within Postgres `max_connections` .
- L59 `QUEUE` uses `useExisting: BullMqQueueAdapter` , so there is one shared instance and not two.
- L66-80 `SECRET_BOX` encrypts stored integration tokens with AES-GCM. Without `PUBLISHING_ENCRYPTION_KEY` it warns and uses a fixed development key; env validation requires a real key in production.
- L81-94 `READINESS_CHECKS` : `SELECT 1` on Postgres and `PING` on Redis. Only `/health/ready` uses them; liveness never checks dependencies.

prisma-persistence.module.ts

- Binds every repository port to its Prisma class (L96-135): users, sessions, password resets, church, uploads, sermons, transcripts, intelligence, embeddings, content, calendar, generations, usage, subscriptions, integrations, publishing, library, analytics, brand, media assets, social and reels.
- L140-142 exports every token automatically from the same list, so adding a repository is one line in `PRISMA_REPOSITORIES` .

external-providers.module.ts

- The comment at L99 sums it up: "swapping a vendor happens only here". Each provider is a factory that reads config and chooses the real adapter or a fake.
- `STORAGE` uses `S3StorageAdapter` (works with AWS S3, Cloudflare R2 and local RustFS). `EMAIL_SENDER` uses Resend, or an outbox adapter that writes emails to a folder in development.
- L116-142 `LANGUAGE_MODEL` is OpenAI or a deterministic fake, **wrapped in `MeteredLanguageModel`** , so every AI call records tokens, estimated cost, the usage meter and metrics.
- L143-173 `TRANSCRIPTION` is OpenAI Whisper or a fake, and records cost per minute of audio per church and sermon.

- Also bound here: the ffmpeg media processor, document text extraction, embedding search, grounded answers, Paystack or fake billing, the OAuth providers (Meta, WhatsApp, Mailchimp, YouTube), publishers, brand and reel renderers, and `SPEAKER_TRACKER` (the Python tracker, or a centre crop).
- L296-313 `SOCIAL_TOKEN_CIPHER`: AES-GCM with a current key plus previous keys, which allows key rotation.
- L324-385 `SOCIAL_PROVIDER_REGISTRY`: Meta, TikTok, YouTube, X and LinkedIn adapters. Each gets `SOCIAL_HTTP_TIMEOUT_MS` and makes one attempt; the queue owns retries.

5. src/shared: the toolbox every module uses

A church never sees `src/shared`, but everything it does passes through it. It holds three kinds of things: **safety nets** (one error format, double-click protection, rate limits, an hourly AI budget per church), **plumbing** (database, Redis, queues, storage, email, the AI client, encryption) and **shared rules** (Bible reference checking, Lagos time, pagination, ids).

Folder	Plain meaning	May import
`shared/domain/`	Pure rules and helpers	Only Node built-ins
`shared/application/`	**Ports** (interfaces such as "I need a cache that can do X") and small helpers for use cases	domain
`shared/infrastructure/`	**Adapters** (Redis, BullMQ, S3, OpenAI, Prisma) plus HTTP glue (pipes, filters, interceptors)	everything

Background job flow: a use case calls `QueuePort.enqueue()`, which goes to `BullMqQueueAdapter` (retry policy from `queue-policies.ts`) and then Redis. In the worker, `BullMqWorkerHost.process()` finds the handler in `JobHandlerRegistry` and calls `handler.handle(payload, JobContext{signal, attempt})`. The handler calls a use case, which passes `signal` down to its I/O. Recurring jobs also take a Redis lock and a fencing token first.

AI call flow: a service calls `LanguageModelPort.generateStructured()`. That goes through `MeteredLanguageModel` (opens a token-counting scope), `OpenAILanguageModelAdapter`, `DependencyGuard.run()`, then `openAIRequest()` with

timeout and abort. Token counts are reported back, the output is checked with Zod, and the meter writes an AI cost row, a usage event and Prometheus metrics.

Config at boot: `ConfigModule.register()` validates against `envSchema`, `loadAppConfig()` reads required keys with `getOrThrow`, `withDerivedSettings()` adds computed flags, and the result is injected everywhere as `APP_CONFIG`.

shared/domain: pure rules

domain-error.ts

- The one error type business code throws (L17-27). Its `kind` (L5-13) is transport-neutral, such as `NOT_FOUND` or `RATE_LIMITED`. The HTTP layer turns it into a status code, so domain code never knows about HTTP.
- `code` is the machine-readable string the frontend switches on. `fieldErrors` carries per-field messages for forms. The helpers `notFound()`, `validationError()` and `notReady()` keep messages consistent.

id.ts

- `newId()` returns a random UUID. Ids are not sequential, so nobody can guess another church's records.
- `deterministicId(seed)` hashes a seed into a UUID-shaped id. A retried job computes the **same** id and finds what an earlier attempt already wrote. This is a key trick for keeping jobs idempotent.

lagos-time.ts

- Version 1 is Nigeria-only. Lagos is UTC+1 with no daylight saving, so a fixed offset is exact and doesn't depend on the server's timezone.
- `lagosMonthRange` gives the UTC start and end of a Lagos calendar month, used for monthly plan periods and quotas. `lagosYear` / `lagosYearRange` do the same for years.

pagination.ts

- `DEFAULT_PAGE_SIZE = 20`, `MAX_PAGE_SIZE = 100`: the hard cap every list endpoint must have. `Page<T>` is what repositories return; `skipFor()` turns a page number into an SQL offset.

text.ts

- `normalizeForComparison` lower-cases text and strips punctuation and curly quotes.
- `isVerbatimExcerpt` checks that an AI "quote" really appears in the transcript. It stops the model from inventing quotes the pastor never said.

- `slugify` strips accents and keeps a-z, 0-9 and dashes, up to 60 characters (for church slugs).

scripture/bible-books.ts

- A data table of the 66 books: `name`, lower-case `aliases` ("gen", "1 cor") and `verses` (the verse count for each chapter). Where translations differ by one verse, the larger count is used so a real verse is never rejected. Edit it only to add an alias.

scripture/scripture.ts

- `findBook` normalises "First Corinthians", "1 Cor.", "II Kings" and "the book of…" and looks the book up in a pre-built index.
- `verifyReference` checks chapter and verse against the real counts. Its comment says it all: it "never trusts a model's claim that a verse exists".
- `formatReference` builds the display string ("Psalm 23", "John 3:16-18"). `parseReference` reads one written or spoken reference ("First John chapter 4 verse 8").
- `detectReferences` scans transcript text. It only uses book names of 4+ letters and requires a capital letter, to avoid false hits such as "mark 5". Only verified references are returned.

scripture.spec.ts, shared-domain.spec.ts

- Tests that the table has 66 books and 1,189 chapters, plus parsing, bounds checks and detection. Also tests the Lagos month boundary (23:30 UTC on 31 October is already November in Lagos), quote matching and slugify.

shared/application: ports and helpers

ai-budget.ts

- `AI_BUDGETS` sets hourly ceilings per church: 120 edits, 30 generate-week runs, 60 reel hooks, 10 reel candidate runs.
- `assertAIBudget` increments a Redis counter `ai-budget:<kind>:<churchId>` and throws `RATE_LIMITED / AI_LIMIT_REACHED` when over. Per-IP throttles can be dodged by using many IPs; this caps spending per **church**.

auth-context.ts

- `AuthContext` says who is calling: `userId`, `sessionId`, `churchId`. `churchId` is always derived on the server.

- `CHURCH_DIRECTORY` is a read-only port that lets other modules look up a user's church. The church module implements it, which avoids import cycles.

church-profile.ts

- `CHURCH_PROFILE` port: tone, Bible translation, audience and words to avoid. These shape the AI prompts.

jobs.ts

- `JobContext` is what every job handler receives: attempt numbers and an `AbortSignal` that fires on time limit, shutdown or a lost lock. Recurring jobs also get `fencingToken` and `assertCurrent()`.
- `JobHandler` is the contract (queue + jobName + handle). `PermanentJobError` means "retrying cannot help", so the worker stops retrying.
- `checkpoint(guard)` is called between batches to stop if the job was aborted or superseded. `JobHandlerRegistry` maps `queue:jobName` to a handler; `defineJobHandler` is a typed shortcut.

reel-jobs.ts

- The job contract between the sermon pipeline (producer) and the reels module (consumer). It lives in shared so neither module imports the other. `reelCandidatesJobId` builds a deterministic job id, so the same job is never queued twice.

ports/

- `cache.port.ts`: get and set with TTL, `setIfAbsent` (atomic "only if nobody else has", used for locks), `take` (read and delete, for single-use tokens such as OAuth state), `increment` (fixed-window counters).
- `clock.port.ts`: `now()`, injected so tests can freeze or advance time.
- `email.port.ts`: `send({to, subject, text, html?})`.
- `language-model.port.ts`: `generateStructured` (JSON matching a Zod schema) and `embed` (vectors). `AICallContext` tags each call with church and sermon for cost tracking. `EMBEDDING_DIMENSIONS = 1536` must match the pgvector column.
- `plan-entitlements.port.ts`: what the church's plan allows this period. Only billing implements it, so plan rules live in one place.
- `queue.port.ts`: `QUEUES` lists every queue name. `enqueue` takes `jobId` (dedupe), `delayMs` and `priority`.
- `secret-box.port.ts`: `seal` / `open` to encrypt integration tokens at rest. `TokenGeneratorPort` creates random secrets whose **hash** is stored.

- `storage.port.ts`: presigned upload and download URLs, delivery URLs for social networks, `putObject` (overwrites, so retried renders are idempotent), `headObject`, `downloadToFile` (streams to disk instead of loading into memory), `deleteObject`. Only `churches/<id>/sermons/<id>/generated/…` may go through the public CDN.
- `unit-of-work.port.ts`: `run(fn)` wraps everything inside in one DB transaction. `runLocked(key, fn)` adds a lock, so two concurrent requests cannot both pass a "check, then write" (for example a plan limit).
- `usage-meter.port.ts`: `record` with a `dedupeKey`, so a retried job counts once and never double-bills. `AIUsageRecorderPort` writes AI cost rows.

shared/infrastructure: adapters and HTTP glue

ai/openai-http.ts

- `openAIRequest` makes one POST to OpenAI using built-in `fetch`. The caller's signal is combined with `AbortSignal.timeout(timeoutMs)`, so a timed-out call really stops.
- It **never retries**: the queue or the caller is the single retry layer. Error bodies are trimmed to 300 characters because they can echo our input. Only 429, 5xx and network errors count against the circuit breaker.

ai/openai-language-model.adapter.ts

- A `DependencyGuard` (max 16 calls in flight; opens after 5 failures for 30 s) protects the process from a slow provider.
- `generateStructured` uses `response_format: json_schema, strict: true` with a 120 s timeout, then **Zod-validates** the result. Model output is never trusted.
- `embed` uses a 60 s timeout and sorts results by index so vectors line up with their inputs.

ai/metered-language-model.ts

- A wrapper around any `LanguageModelPort`. Every call records an AI cost row, a usage event and Prometheus metrics. Cost = tokens × per-million price from config.
- It uses `Promise.allSettled`, so a metering failure never breaks the actual operation. Prompts and responses are never logged.

ai/token-usage.ts, strict-json-schema.ts, fake-language-model.ts

- `token-usage.ts` uses `AsyncLocalStorage` to pass token counts from the adapter back to the meter without changing the port.
- `strict-json-schema.ts` converts a Zod schema into the JSON Schema subset OpenAI strict mode accepts. Limits it has to drop are still enforced afterwards by Zod.

- `fake-language-model.ts` is a deterministic fake for development and tests (forbidden in production). It builds outputs from the real input. `embedText` makes hashed bag-of-words vectors, so similar texts really are close in search tests.

cache/redis-cache.adapter.ts

- Every key is prefixed `abyde:` and values are JSON. `setIfAbsent` uses `SET NX` (atomic across replicas); `take` uses `GETDEL`.
- `increment` uses `MULTI` with `INCR` + `EXPIRE NX`, so the window's TTL is set once and not extended on every hit.

config/env.schema.ts

- The Zod schema for **every** environment variable, with helpers `booleanString`, `optionalString` (blank becomes undefined) and `csv`.
- `superRefine` holds cross-field rules. Its local helper `require(cond, key, msg)` adds an error **when `cond` is true**; read it as "fail if". Examples: an OpenAI key is needed when the provider is `openai`; production forbids fakes, the outbox and Swagger; staging and production need public https URLs, secure cookies, a metrics token of 24+ characters and encryption keys.
- `withDerivedSettings` adds `swaggerEnabled` and `cookieSecure`.

config/config.module.ts

- A global module. `register()` runs `ConfigModule.forRoot({ validationSchema: envSchema })`; `.env` files are ignored in production and tests. `loadAppConfig` uses `getOrThrow` for required keys. Nothing is logged.

crypto/

- `aes-gcm-secret-box.ts`: AES-256-GCM for integration credentials, stored as `v1.<iv>.<tag>.<ciphertext>`. Each seal gets a fresh random IV, and the GCM tag makes tampered data fail to decrypt.
- `aes-gcm-token-cipher.ts`: a stronger version for social tokens. **Associated data** (church + connection) means a token only decrypts where it was sealed, and a **key id** supports key rotation. It is the only shared file that imports from a module (`modules/social`).
- `random-token-generator.ts`: 32 random bytes as base64url; `hash` is SHA-256 hex.

email/

- `resend-email.adapter.ts`: sends through Resend's HTTP API with a 15 s timeout inside a `DependencyGuard` (max 8 in flight). Non-OK responses throw so the email job retries.
- `outbox-email.adapter.ts`: development only. Writes each email to a file (mode `0o600`), so you can click reset links locally without them appearing in logs.

http/auth.decorators.ts

- `@Public()` / `@OptionalAuth()` set metadata the global auth guard reads (auth is deny-by-default).
- `@CurrentChurchId()` is the **only** way controllers get `churchId`, from the session and never from the body. It returns 401 if you are not signed in and 403 `ONBOARDING_REQUIRED` if you have no church yet. `@CurrentAuth()` gives the full context.

http/error-envelope.filter.ts

- A global filter that extends `BaseExceptionFilter`. `toEnvelope` maps `DomainError` (via `KIND_STATUS`), `HttpException`, `DependencyUnavailableError` (503 + `Retry-After`) and bad or oversized JSON (400/413).
- Anything unknown is reported to Sentry and passed to the base filter as an opaque 500 ("Something went wrong."). Stack traces and SQL never reach clients.

http/idempotency.interceptor.ts

- Implements the idempotency contract, stored in Redis. It applies only to POST, PATCH, PUT and DELETE. `@RequireIdempotencyKey()` makes the header mandatory.
- The stored key is a hash of the user (or IP) + method + URL + header value, so it is scoped per user. A body fingerprint catches reuse with a different body (422).
- A duplicate that arrives while the first is still running gets 409 + `Retry-After: 2` (120 s lock). Successes and 4xx responses are stored for 24 h; 5xx responses are not, so they can be retried. Password routes are never stored.

http/: pagination.ts, redis-throttler.storage.ts, route-limits.ts, uuid.pipe.ts, zod-validation.pipe.ts

- `pagination.ts`: `pageQuery` gives Zod fields for `?page=&pageSize=` (default 20, max 100); `toPaginated` builds `{data, meta}`.
- `redis-throttler.storage.ts`: throttler counters in Redis, shared across replicas, using atomic `MULTI` (INCR + PEXPIRE NX + PTTL).

- `route-limits.ts`: `CREDENTIAL_THROTTLE` (10/min) and `AI_THROTTLE` (20/min), read at request time from settings `bootstrap.ts` stores.
- `uuid.pipe.ts`: `ParseIdPipe` turns a non-UUID id into a plain 404, giving no hint about what valid ids look like.
- `zod-validation.pipe.ts`: `@Body(zod(schema))` runs `safeParse` and throws a 422 `VALIDATION_ERROR` with `{field: [messages]}`.

locks/

- `redis-lock.store.ts`: the lock store for `@nestjs/locks`, in Redis. Lua scripts run atomically: ACQUIRE is `SET NX PX` and, if it wins, increments a counter that becomes the **fencing token**. RENEW and RELEASE act only if you still own the lock.
- `fencing-registry.ts`: remembers the highest token per lock. If an old holder paused (GC, network) and wakes after someone else took over, `assertCurrent` throws `StaleLockHolderError` before it can write.

observability/

- `error-reporter.ts`: a Sentry wrapper, active only when a DSN is set. `beforeSend` deletes bodies, cookies, query strings and auth headers.
- `fatal-errors.ts`: `crashOnFatalErrors()` writes one JSON log line with the stack, reports to Sentry, then calls `process.exit(1)`.
- `metrics.service.ts`: Prometheus metrics for HTTP latency, jobs, queue depth and AI calls and cost, plus Node defaults.
- `http-metrics.interceptor.ts`: times every request except health and metrics. It labels by `Controller.handler`, not the raw URL, to keep the number of series small.

prisma/

- `prisma.service.ts`: the Prisma client on `@prisma/adapter-pg`, with an explicit pool size, a 5 s connect timeout, a Postgres `statement_timeout` and an idle-in-transaction timeout. The `db` getter returns the current transaction (from `AsyncLocalStorage`) or the plain client, so repositories join a transaction automatically.
- `prisma-unit-of-work.ts`: `run` starts `$transaction` (5 s max wait, 20 s timeout). `runLocked` also takes `pg_advisory_xact_lock(hash(key))`, which is released on commit or rollback.
- `connection-budget.check.ts`: at boot, checks pool × processes ≤ `max_connections` − 5. It throws in staging and production and warns elsewhere.

process/run-process.ts

- Runs ffmpeg, ffprobe or the speaker tracker with `spawn` and no shell, so there is no shell injection. It sends SIGKILL on timeout or abort, and caps stderr at 2,000 characters.

queue/

- `queue-policies.ts`: one row per queue with `attempts`, `backoffMs` (exponential), `timeoutMs` per attempt, `discardOnComplete` (emails with single-use links are deleted after success) and `maxConcurrency` (video renders: 1 per process, because ffmpeg uses every core). Every timeout must fit inside `WORKER_SHUTDOWN_GRACE_MS`.
- `bullmq-connection.ts`: Redis options for BullMQ (TLS for `rediss:`), with `maxRetriesPerRequest: null` for its blocking commands.
- `bullmq-queue.adapter.ts`: creates one `Queue` per name, lazily, with explicit `defaultJobOptions` (attempts, exponential backoff, keep completed jobs 24 h, failed jobs 7 days). `:` in job ids is replaced with `.` because BullMQ reserves it.
- `bullmq-worker.host.ts`: one `Worker` per queue with handlers. Recurring schedulers are upserted by id, so N replicas still produce one schedule. Recurring jobs run inside a Redis lock with fencing, and a tick is skipped if the previous run still holds the lock. `PermanentJobError` becomes `UnrecoverableError`. On shutdown it waits for the grace period, then aborts.

redis/redis.provider.ts

- One shared `ioredis` connection (BullMQ has its own), with 5 s connect and command timeouts and at most 2 retries. Connection errors surface through the readiness check rather than crashing the process.

resilience/dependency-guard.ts

- A circuit breaker plus a concurrency cap for one external service. Closed → open after N failures in a row → half-open after `openMs`, where one trial call decides. While open or at capacity it throws `DependencyUnavailableError` straight away (503 + `Retry-After`). State is per process.

storage/s3-storage.adapter.ts

- Works with AWS S3, R2 or RustFS; buckets stay private. SDK `maxAttempts: 1` keeps a single retry layer.
- `createUploadUrl` signs **Content-Type and Content-Length**, so a browser cannot upload a different type or a bigger file. `putObject` and `downloadToFile` stream to and from disk, so large videos never sit in memory.

6. Auth module (src/modules/auth)

Auth handles sign-up, login, the session cookie on every request, password reset, account settings, onboarding and clearing out expired sessions. It is **deny-by-default**: every route in the app needs a session unless it is marked `@Public()`.

How it flows

- **Every request:** `SessionAuthGuard` (global `APP_GUARD`). If the route is `@Public()`, allow it. Otherwise read the cookie and call `ResolveSessionUseCase` (Redis cache, then the database). On success it sets `request.auth = { userId, sessionId, churchId }`; otherwise 401.
- **Login:** controller → `LoginUseCase` (lockout check, user lookup, constant-time password check) → `SessionIssuer` (random token, store only its hash) → `httpOnly` cookie.
- **Register:** controller → `RegisterUseCase` → password policy + normalised email → enumeration limiter → `User.register` → save → `SessionIssuer.issue` → cookie.
- **Forgot password:** controller → `ForgotPasswordUseCase.execute` (one cache write + one enqueue) → worker → `ForgotPasswordUseCase.process` → token row + `email/send` job.
- **Reset:** controller → `ResetPasswordUseCase` → check token → in one transaction: consume the token, save the user, revoke all sessions → evict them from the cache.
- **Purge:** the daily `purge-expired-auth` job → `PurgeExpiredAuthUseCase`.

The login flow, step by step

1. **`POST /auth/login`** in `auth.controller.ts` (L122). The route is `@Public()` with the tighter credential throttle. `LoginBody` (Zod) caps the email at 254 and the password at 256 characters, so nobody can send a giant password to waste hashing time. The controller adds `userAgent` and `ip`, calls the use case, sets the cookie and returns `me`.
2. **`LoginUseCase`** (`login.use-case.ts`):
   a. Trims and lower-cases the email (L54).
   b. **Lockout** (L57-68): two Redis counters, `login-failures:{email}:{ip}` (10 failures per 15 min) and `login-failures:{email}` (50 across all IPs). Per email + IP, so an attacker cannot lock the real owner out; the overall ceiling still stops a botnet.
   c. Looks up the user through `UserRepositoryPort` (L69).
   d. **Always verifies a hash**, even for an unknown email: `user?.passwordHash ?? DUMMY_HASH` (L70-73). Otherwise a missing email would answer in 1 ms instead of ~100 ms, telling an attacker which emails exist.

   e. On failure, increments both counters and throws the same `INVALID_CREDENTIALS` for a wrong email or a wrong password.
   f. On success, clears the per-IP counter, issues a session and returns `toMeView(...)`, an explicit DTO that can never include the password hash.
3. **`SessionIssuer.issue`** (`session-issuer.ts`): lifetime is days with "remember me", hours without. The token is 32 random bytes; **only its SHA-256 hash is stored**, so a leaked database cannot be used to log in. Time comes from `ClockPort`, so tests can control it.
4. **The cookie** (`session-cookie.ts`): `httpOnly` (JavaScript can't read it, so XSS cannot steal it), `secure` (HTTPS only), `sameSite: 'lax'` (not sent on cross-site POSTs, the main CSRF defence).

File by file

auth.module.ts

- Builds `AUTH_SETTINGS` and `SESSION_COOKIE_SETTINGS` from validated config (L38-56), so use cases receive plain settings rather than the whole config.
- Binds `PASSWORD_HASHER` to scrypt (L57). `{ provide: APP_GUARD, useClass: SessionAuthGuard }` (L71) makes auth deny-by-default across the whole app.
- Exports `GetMeUseCase`, `CompleteOnboardingUseCase` and `ResolveSessionUseCase` for the church module and others.

domain/email.ts, password-policy.ts, user.entity.ts

- `normalizeEmail` trims, lower-cases, checks the format and a 254-character maximum, so `A@B.com` and `a@b.com` are the same account.
- The password policy is 8-128 characters with at least one letter and one digit. The maximum stops very long inputs that would make hashing slow.
- `User.register()` validates the name (1-120 characters). `changeEmail` returns `false` if nothing changed and clears `emailVerifiedAt` when it does. `changePasswordHash` and `completeOnboarding` are the only ways to change those fields.

application/auth-settings.ts, me.ts, session-cache.ts

- `auth-settings.ts`: session lifetime, "remember me" lifetime, reset-link lifetime and web URL.
- `me.ts`: `MeView` is what `/auth/me` returns (id, name, email, church). `IssuedSession` hands the token to the HTTP layer; use cases never touch cookies.

- `session-cache.ts`: Redis keys for sessions (cached 60 s), revocation "tombstones" and user → churchId. `evictSessions` writes a tombstone (120 s) **before** deleting the cache entry. Without it, a request that read the session just before logout could put it back in the cache.

application/ports/out/

- `user-repository.port.ts`: `findById`, `findByEmail`, and `save` (throws `EMAIL_TAKEN` on a duplicate).
- `session-repository.port.ts`: sessions (create, find by hash, touch, revoke, `revokeAllForUser`, purge), password reset tokens (`consume` returns `false` if another request already used it) and `PasswordHasherPort`. Only token **hashes** are stored.

use-cases/resolve-session.use-case.ts

- Rejects tokens of the wrong length before doing any work (L47). Hashes the token, tries Redis, and falls back to the database, ignoring revoked or expired sessions.
- Updates `lastSeenAt` at most every 10 minutes, so there is no database write on every request.
- If this request filled the cache but a tombstone exists, it undoes the fill and returns `null` (the race fix above).
- The user's `churchId` is cached for 1 h, but only positive results, so a newly created church shows up at once.

use-cases/register.use-case.ts

- Checks the password policy and normalises the email first.
- **Enumeration limiter** (L61-77): the web app needs an explicit `EMAIL_TAKEN`, which reveals that an account exists, so each IP can receive that answer only 5 times an hour.
- Hashes, saves and issues a session. If two sign-ups race, the unique email index makes the second one get `EMAIL_TAKEN`.

use-cases/account.use-cases.ts

- `GetMeUseCase` returns `SIGNED_OUT` when there is no session.
- `LogoutUseCase` is idempotent: unknown or already-revoked tokens succeed silently, and the cache is always evicted.
- `UpdateProfileUseCase`: changing the email requires the **current password**, so a borrowed session cannot take over the account. After an email change it invalidates reset links, signs out other devices and emails the **old** address.

- `ChangePasswordUseCase`: verifies the old password, saves the new hash, invalidates reset links and revokes every other session.

use-cases/password-reset.use-cases.ts

- `ForgotPasswordUseCase.execute` does identical work for every email (one `setIfAbsent`, at most 1 per address per minute, and one enqueue), so its response and timing never reveal whether an account exists.
- `process` runs in the worker: invalidates old tokens, creates a random token (stores the hash), builds an expiring link and enqueues the email.
- `ResetPasswordUseCase`: checks hash, expiry and single use. In one transaction it consumes the token (only one concurrent request can win), saves the new hash and revokes **all** sessions.

use-cases/complete-onboarding, purge-expired-auth

- `CompleteOnboardingUseCase` is called by church creation and does nothing if already done (idempotent).
- `PurgeExpiredAuthUseCase` deletes sessions and reset tokens that expired more than 30 days ago. The 30 days keep recent sign-in history for support.

infrastructure/scrypt-password-hasher.ts

- scrypt with N=2^15, stored as `scrypt$N$r$p$salt$hash`. `verify` always runs the key derivation, even for malformed hashes, and compares with `timingSafeEqual`, so timing never leaks anything.

infrastructure/persistence/prisma-auth.repositories.ts

- `PrismaUserRepository`: upsert by id; a unique violation becomes `EMAIL_TAKEN`.
- `PrismaSessionRepository`: `touch` and `revoke` use `updateMany … where revokedAt: null`, so they are safe to repeat and never un-revoke a session. `revokeAllForUser` returns the token hashes in one query.
- `PrismaPasswordResetRepository.consume` is an atomic `updateMany where usedAt: null` that returns `count === 1`. That is what makes reset links single-use under concurrency.
- These tables are keyed by user rather than church, because auth comes before a church exists.

presentation/http/

- `auth.controller.ts`: register, login, logout, me, update profile, change password, forgot and reset. Each validates with Zod and calls one use case.

- `session-auth.guard.ts`: lets non-HTTP contexts (queue jobs) and `@Public()` routes through; otherwise resolves the cookie into `request.auth`. `@OptionalAuth()` routes pass even when signed out.
- `session-cookie.ts`: set, clear and read the cookie with the options above.

presentation/queue/auth-job.handlers.ts

- Registers the `password-reset-request` job and the daily `purge-expired-auth` job with its recurring schedule.

7. The business modules, file by file

Each module below follows the same four-layer pattern. For each one you get a short "how it flows" chain, then every file. They are ordered the way a sermon travels: upload, processing, library, content, brand, reels, publishing, social, then the supporting modules (billing, church, analytics, usage, health, system).

7.1 Uploads (src/modules/uploads)

Uploads lets the browser send big files straight to object storage, so they never pass through our server.

How it flows: `POST /uploads/presign` → Zod → `PresignUploadUseCase` → `upload-policy.ts` checks type, extension and size → `StoragePort.createUploadUrl` → save an `uploads` row → return `uploadUrl`, `headers` and `objectKey`. The browser PUTs the file directly to storage. Later, `POST /sermons` with that `objectKey` calls `ConsumeUploadUseCase.verify`, then `.claim` (used exactly once). A maintenance job deletes uploads that were never used within 24 h.

uploads.module.ts

- Registers the controller and three use cases, and exports them so `SermonsModule` can use `ConsumeUploadUseCase`. Adapters are bound in `src/composition/`.

domain/upload-policy.ts

- `POLICIES` (L20-45) is an **allow-list** of MIME types per purpose, each with allowed extensions, a kind and a size cap. Sermon media can be up to 2 GB. Logos must be PNG, JPEG or WebP up to 5 MB. **SVG is refused** because an SVG can carry scripts.
- `validateUploadRequest` (L59-92) normalises the content type, rejects types not on the list, rejects an extension that doesn't match the type (stops `virus.exe` posing as `audio/mpeg`), and rejects a size of 0 or over the cap.

- `objectKeyFor` builds keys like `churches/<churchId>/sermons/<uuid>.mp3`; the random part means keys can't be guessed. `isKeyOwnedBy` is a quick tenancy check on the prefix.

domain/upload-policy.spec.ts

- Tests allowed types, bad types, wrong extensions, oversize and empty files, SVG logos, and church-scoped keys.

application/ports/out/upload-repository.port.ts

- `markConsumed` and `deleteIfUnconsumed` return booleans and only succeed if nobody else got there first, so two requests can never both use the same upload.

application/use-cases/upload.use-cases.ts

- `PresignUploadUseCase`: validates, makes a church-scoped key, asks storage for a signed PUT URL that expires in 15 min, and records the upload.
- `ConsumeUploadUseCase.verify` **never trusts the key from the browser**. It checks the prefix belongs to this church, that a matching unused row exists for this purpose, and (with `headObject`) that the file really exists with the right size and type. An oversize file is deleted. `verify` writes nothing, so callers run it **before** opening a transaction and a slow network call never holds a DB lock.
- `claim` marks the upload used, exactly once, inside the caller's transaction. If the sermon insert rolls back, the upload is free again.
- `PurgeStaleUploadsUseCase`: up to 500 unused uploads older than 24 h per run. It deletes the **DB row first**, then the file, so nobody can attach the key in between. `checkpoint(guard)` stops the loop on shutdown or a lost lock.

infrastructure/persistence/prisma-upload.repository.ts

- Sizes are stored as `BigInt` (files can exceed 2^31 bytes). `markConsumed` / `deleteIfUnconsumed` use `updateMany` / `deleteMany` with `consumedAt: null` in the `where` and check `count === 1`: an atomic compare-and-set done by the database.

presentation/http/uploads.controller.ts, presigned-upload.response.ts

- Zod validates `fileName`, `contentType` and `sizeBytes`; this route only accepts `purpose: 'SERMON_MEDIA'`. `churchId` comes from the session. The response DTO copies only what the client needs.

7.2 Sermons (src/modules/sermons)

The heart of the app. A sermon (audio, video, PDF, DOCX, TXT or pasted text) becomes a transcript of timestamped **segments**, an AI summary (themes, key points, real quotes, stories, applications, prayers, questions, Bible references) and **embeddings** for search.

HTTP flow (API): `POST /sermons` → Zod → `CreateSermonUseCase` → `ConsumeUploadUseCase.verify` → per-church lock (`uow.runLocked`): quota check, claim the upload, `Sermon.create`, insert, save the asset or pasted text, record usage → enqueue a `start` job → **202** `{ id }`. Other routes: list, get, patch, delete, status, retry, transcript and intelligence. Each calls exactly one use case.

Pipeline flow (worker): `start` → `StartSermonProcessingUseCase` picks the phase to resume from → `transcribe` (download to scratch space; ffmpeg chunks media and each chunk goes to speech-to-text, or documents are extracted; `segmentTranscript` splits; save) → `analyze` (one schema-checked AI call plus guardrails; save scriptures and intelligence) → `index` (embed segments 64 at a time into pgvector; mark READY; queue Reel candidates, best effort). Maintenance: `recover-stuck-sermons` every 10 min and `purge-expired-sermon-sources` every hour.

sermons.module.ts

- Imports `UploadsModule` and registers the controller, every use case, the analysis service and queue handlers. Only `GetSermonUseCase` is exported; other modules reach sermons through ports.

application/sermon-jobs.ts

- `SermonJobPayload` holds `churchId`, `sermonId`, `attempt` (the processing run number) and an optional `fromStage`. Each phase has its own queue, so each is retried separately.
- `sermonJobId` builds `sermon:<id>:<attempt>:<phase>`. The same job always gets the same id, so enqueueing twice does nothing the second time.

domain/processing.ts

- `SermonStatus` is what the user sees; `PROCESSING_STAGES` are the seven steps the web app shows. `statusForStage` and `phaseForStage` let a retry know where to resume. `STAGE_FAILURE_MESSAGES` are friendly messages; raw provider errors are never shown.

domain/sermon.entity.ts

- `create` always starts at `QUEUED` / `UPLOAD` with `processingAttempts = 1`. `restore` rebuilds from the DB and copies arrays so nobody can change them by accident.

- `updateDetails` checks every field (title 200, speaker 120, real date, tags lower-cased, de-duplicated, max 20 of 40 characters). It builds the new values first and swaps them in at the end, so a failure changes nothing.
- `acceptsJobFor(attempt)` is the key to **stale-job safety**: a job only runs if its attempt matches the current run and the sermon isn't READY, FAILED or ARCHIVED.
- `retry` works only when FAILED with `canRetry`. It bumps `processingAttempts`, so any old jobs still in the queue are ignored.

domain/sermon-intelligence.ts, transcript-segmenter.ts (+ specs)

- `IntelligenceItem` links every extracted item to its segment and timestamps, so the evidence can be checked.
- `estimateTiming` gives documents fake timestamps at about 400 ms per word, so every segment still has a time.
- `segmentTranscript` cuts a segment once it is 45 s+ **and** ends a sentence, or at the hard limit of 120 s or 280 words. Word timings are kept only if every fragment has them, because Reel captions need full coverage.

application/ports/out/

- `sermon-repository.port.ts`: every method takes `churchId` (except two cross-church maintenance methods). Three separate writes: `updateDetails` (user fields), `updateProcessing` (processing fields, only while in the same run) and `updateForRetry`. A worker can therefore never overwrite a user's edit or bring a deleted sermon back.
- `transcript-repository.port.ts`: `replace` swaps the whole transcript, so running it twice is safe. `listSegments` leaves out word timings; `listSegmentsWithWords` includes them. The embedding port has a vector search and a keyword fallback.
- `processing.ports.ts`: `TranscriptionPort`, `MediaProcessorPort`, `DocumentTextExtractorPort`, `SermonQuotaPort` (from billing), `ScratchSpacePort`. Every slow call takes an `AbortSignal`.

application/use-cases/sermon-crud.use-cases.ts

- `CreateSermonUseCase`: exactly one of `objectKey` or `transcriptText` (pasted text 200 to 1,000,000 characters). `verify` runs **before** the lock. Inside `runLocked('sermon:create:<churchId>')` it checks quota, claims, inserts, adds the asset (with `expiresAt` from the plan's retention days) and records usage with dedupe key `sermon:<id>`. The lock stops two simultaneous uploads from both slipping under the plan limit. If enqueueing fails, the sermon is marked FAILED with retry allowed instead of hanging on "queued" forever.

- `DeleteSermonUseCase` deletes the row first (cascades), then the files. A failed file delete only logs, so you can end up with an extra file but never a row pointing at a missing file.
- `RetrySermonUseCase` takes a per-sermon lock, calls `retry()` and `updateForRetry` (a conflict means someone else already retried), then enqueues `start` with the new attempt. Transcript and intelligence return `notReady()` until READY.

application/use-cases/sermon-pipeline.use-cases.ts

- `SermonPipelineSupport` wraps every phase: `load` skips the job if `acceptsJobFor` is false; `persist` throws `SermonSupersededError` if the guarded write matched no row; `run` stops quietly on Superseded, rethrows other errors so the queue retries, and on the final attempt or a `PermanentJobError` marks the sermon FAILED with a friendly message.
- `TranscribeSermonUseCase`: skips if segments already exist (crash-safe), refuses transcripts under 50 words, saves transcript and state in one transaction, records minutes with dedupe key `transcription:<id>:<attempt>`. It re-checks the stored file size, downloads to scratch space, and sends chunks **one at a time** to bound memory and provider calls.
- `AnalyzeSermonUseCase`: calls the analysis service with the church's AI profile and the job's `signal`, replaces scripture rows, then in one transaction saves intelligence and copies summary, themes and scriptures onto the sermon for search.
- `IndexSermonUseCase`: embeds `title + segment text` in batches of 64, replaces embeddings and marks READY in one transaction, then tries to queue Reel candidates (best effort).

use-cases/recover-stuck-sermons, purge-expired-sources

- Recover: finds sermons stuck "processing" for 2 hours (worker died, queue down), up to 100 per run. Inside a per-sermon lock it re-reads and re-checks `acceptsJobFor`, then marks FAILED with retry allowed, so users never see an endless spinner.
- Purge: deletes original uploads past the plan's retention period (up to 200 per run), keeping everything made from them. It skips sermons still processing.

application/services/sermon-analysis.service.ts, prompt-context.ts

- `SermonAnalysisSchema` (Zod) limits every field the AI returns. `analyze` uses `temperature: 0.2`, the shared `GUARDRAILS`, numbered segments, church name and audience.
- `assembleIntelligence` maps items to real segments. **Quotes are kept only if they appear word for word** in their segment.

- `collectScriptures` merges the church's main scripture, regex-detected references and the model's suggestions. Only references valid against the Bible table get `verified: true`, and only verified ones are shown.
- `buildTranscriptContext` numbers segments as `[n] (time) text`; over 180k characters it keeps an evenly spread sample so the prompt fits.

infrastructure/transcription/

- `openai-transcription.adapter.ts`: `/audio/transcriptions` with segment and word timestamps; a 5-minute timeout per call with the job's `signal`; a `DependencyGuard` (max 4 at once, open 60 s after 4 failures). `attachWords` places each word in the segment containing its midpoint. Cost is always recorded in `finally`.
- `fake-transcription.adapter.ts`: a fixed sample sermon for development and tests.

infrastructure/documents/document-text-extractor.ts

- **mammoth** for DOCX, **unpdf** for PDF, UTF-8 for TXT (BOM removed). Parse limits are 100 MB for PDF/DOCX and 20 MB for TXT; bigger files fail with a friendly `PermanentJobError` rather than an out-of-memory crash.

infrastructure/media/

- `ffmpeg-media-processor.ts`: `ffprobe` / `ffmpeg` via `spawn`, so media stays on disk. `-protocol_whitelist file` stops a malicious playlist making ffmpeg fetch URLs (SSRF). Output is mono 16 kHz 32 kbps MP3 in 20-minute chunks (~5 MB each). Killed on abort or after 30 minutes.
- `os-scratch-space.ts`: a temp folder that is **always** deleted in `finally`, so the worker's disk never fills up.

infrastructure/persistence/

- `prisma-sermon.repositories.ts`: every `where` includes `churchId`. `updateProcessing` adds `processingAttempts` and a status check to the `where` (a guarded write) and returns `count === 1`. List runs `count` and `findMany` in parallel; `contentCounts` uses one `groupBy` per page, avoiding N+1 queries.
- `prisma-transcript.repositories.ts`: word timings are stored as compact `[text, start, end]` tuples (about a third of the size). `replace` deletes and re-creates, inserting 500 segments at a time. The embedding repository uses raw SQL (Prisma has no `vector` type) but every value is a bound parameter via `Prisma.sql`. Vector search filters `church_id` **inside** the search and joins on `status='READY'`.
- `sermon.mapper.ts`: converts rows to the domain `Sermon`, to insert data and to the `SermonView` read model.

- `church-scoping.spec.ts`: a recording fake Prisma (JavaScript `Proxy`) proves **every** query from the sermon, transcript, content and calendar adapters includes `churchId`, and that an SQL-injection string stays a plain parameter.

presentation/

- `sermons.controller.ts`: `CreateSermonBody` uses `z.object` (drops unknown keys); `UpdateSermonBody` uses `.partial().strict()` (rejects them). `POST` returns **202** because work continues in the background. `ParseIdPipe` rejects non-UUID ids.
- `sermon.responses.ts`: response DTOs; dates as ISO strings, segments without word timings.
- `sermon-job.handlers.ts`: registers a handler per pipeline job, each passing the payload to one use case, plus the two recurring maintenance jobs.

7.3 Library (src/modules/library)

Browse, filter, search by meaning, and **Ask**: a question answered only from the church's own sermons, with links to the exact moments.

How it flows: `POST /library/search` → `SearchLibraryUseCase` (READY sermons only). `POST /library/semantic-search` (AI throttle) → embed the query (cached) → segment search (keyword fallback) → best score per sermon → page. `GET /library/facets` → 60 s cache. `POST /ask` → segment search (12 results, similarity ≥ 0.25) → `GroundedAnswerPort` (AI) → answer + cited sources.

library.module.ts, ports/out/

- `SEGMENT_SEARCH`, `GROUNDED_ANSWER` and `LIBRARY_FACETS` are bound in `src/composition/`. `SegmentSearchPort` reports a `mode` (`semantic` or `keyword`) so callers know when the fallback was used.

use-cases/library.use-cases.ts

- `SearchLibraryUseCase` forces `statuses: ['READY']`. `SemanticSearchUseCase` fetches up to 300 matches, drops those under 0.2, keeps each sermon's best score and pages in memory. Facets are cached per church for 60 s; the key includes `churchId`.

use-cases/ask-ministry.use-case.ts

- **RAG** (retrieval-augmented generation): the AI only sees text retrieved from this church's sermons. If nothing scores 0.25+, it returns a fixed "no evidence" answer **without calling the AI**, which saves money and prevents made-up answers. If the model says evidence is insufficient, no sources are returned.

infrastructure/

- `embedding-segment-search.adapter.ts`: caches each query embedding for 24 h under a SHA-256 of the text (the same search isn't billed twice, and the key holds no raw text). If embedding fails, it falls back to keyword search, so search degrades instead of breaking.
- `ai-grounded-answer.adapter.ts`: strict rules (only the numbered excerpts, never invent quotes or verses). If the AI fails, it returns a fallback that points to the first 3 passages.
- `prisma-library-facets.ts`: six `SELECT DISTINCT` queries in parallel, filtered by church and READY. The year adds one hour so it counts in Lagos time.

presentation/http/library.controller.ts

- Zod bodies (year 1900-2100, queries up to 500 characters, questions 3-1000). `@Throttle(AI_THROTTLE)` on semantic search and Ask because they cost AI money.

7.4 Content (src/modules/content)

One click on **Generate My Week** writes devotionals, social posts, prayers, a Bible study, an email, WhatsApp messages, quotes and video ideas from one sermon. The team can edit, approve, rewrite with AI ("shorter", "warmer"), regenerate, view version history and put items on a calendar.

Generate a week: `POST /sermons/:sermonId/generate-week` → `GenerateWeekUseCase` (sermon must be READY, check AI budget, lock per sermon, plan jobs) → write the group + jobs → enqueue one job per content type → worker: `GenerateContentTypeUseCase` → load sermon context + `ContentGenerationService.generate` (AI) → save items in a transaction → job COMPLETED → usage metered. The web app polls `GET /sermons/:sermonId/generations/:groupId` until `done: true`.

Edit / approve / rewrite: `PATCH /content/:id` (or `/approve`, `/rewrite`, `/regenerate`, `/versions/:v/restore`) → use case → `ContentMutations.mutate` (lock `content:{id}`, refuse if being published, call the entity method, remove the calendar entry if needed, save) → `ContentPresenter`.

Calendar: `POST /calendar` → `ScheduleContentUseCase` (lock, create or move the one calendar entry, set SCHEDULED). **Recovery:** every 10 minutes `recover-stuck-generations` fails jobs stuck for 30+ minutes, so the UI can offer "Retry failed".

content.module.ts

- Lists 3 controllers and every use case. Binds `SERMON_CONTEXT` to `SermonContextAdapter` here (it adapts another module, not an outside system). Exports `ContentPresenter`.

application/ai-budget.ts, content-presenter.ts

- `ai-budget.ts` re-exports the shared per-church AI budgets (edit 120/hour, generate-week 30/hour).
- `ContentView` includes a `source` block (sermon, segments, scriptures, `verbatim`) so users see where the AI got the text. `present()` fetches segments and sermons for a whole page in **two batched calls** instead of one query per item (avoids N+1). A segment is shown only if it belongs to the item's sermon.

application/ports/out/

- `content-repository.port.ts`: tokens `CONTENT_REPOSITORY`, `CONTENT_QUERY`, `CALENDAR_REPOSITORY`, `GENERATION_REPOSITORY`. Every method takes `churchId` first. `save()` upserts and appends new versions; `deleteByGenerationJob` lets a retried job wipe its own partial output.
- `sermon-context.port.ts`: everything generation needs from one sermon (title, speaker, intelligence, segments, scriptures), read-only and filtered by church.

infrastructure/sermon-context.adapter.ts

- Implements the port by calling the **sermons module's own repository ports**. Content reads sermons; sermons knows nothing about content. Loads in parallel with `Promise.all`.

domain/content-types.ts

- 8 `CONTENT_TYPES`, 6 statuses, 5 `REWRITE_ACTIONS`. `as const` lets TypeScript and Zod share the same arrays. `SourceMetadata` is the evidence trail; `verbatim: true` means the body is the pastor's exact words.

domain/content-item.entity.ts

- The lifecycle: DRAFT → APPROVED → SCHEDULED → PUBLISHED (or FAILED).
- **Versioning:** every change calls `recordVersion()`. History is append-only and never rewritten.
- `revise()` sets `verbatim` to false if a quote's words change (we never claim edited text is the pastor's exact words), and moves non-DRAFT items back to DRAFT with `unschedule: true`, because a calendar entry belongs to the approved wording.

- `approve()` is idempotent. `schedule()` works only on APPROVED/SCHEDULED items. PUBLISHED and ARCHIVED items are locked. Title 1-200 characters, body 1-20,000.

domain/calendar-item.entity.ts

- Statuses SCHEDULED, PUBLISHING, PUBLISHED, FAILED. `schedule()` and `reschedule()` refuse past times. `assertNotPublishing()` blocks edits while publishing is in progress.

domain/generation-group.ts

- A "generation group" is one Generate My Week click with one job per content type. `planGeneration()` reuses the latest group: re-queue only FAILED types, leave in-flight and completed ones alone, add new types. This makes "Retry failed" and double clicks **idempotent**.

application/services/content-specs.ts

- `CONTENT_SPECS`: per type, how many to write (e.g. 7 devotionals, 5 prayers), the prompt, the tone and a max body length. `REWRITE_INSTRUCTIONS`: one sentence per rewrite button. To change what the AI writes for a type, edit this file only.

application/services/content-generation.service.ts

- Calls the AI only through `LanguageModelPort`. Zod schemas describe the exact JSON with max lengths; bad output never reaches the database.
- `generate()`: one call per type at temperature 0.7, passing `signal` so a timeout really cancels.
- `generateQuotes()`: quotes are **extracted, not written**. It starts from verified intelligence quotes, asks for more at temperature 0.2 if needed, and keeps only lines that match the transcript word for word.
- `systemPrompt()` adds the church's tone, translation, audience and words to avoid, plus the guardrails. `verifiedScripture()` keeps only real canon references.

application/use-cases/generation.use-cases.ts

- `GenerateWeekUseCase`: rejects if the sermon isn't READY, checks the AI budget, and runs inside `uow.runLocked('generation:{sermonId}')`, so two concurrent clicks resolve to one group.
- `GenerateContentTypeUseCase` (worker): skips COMPLETED jobs. In **one transaction** it deletes partial output from an earlier attempt, saves the new items and marks the job COMPLETED, which makes retries safe. On error: RETRYING, or FAILED if permanent

or the last attempt, then rethrow. Metering runs after, with a `dedupeKey`, so a metering failure can never trigger a retry that deletes delivered content.

application/use-cases/content.use-cases.ts

- `ContentMutations.mutate()`: per-item lock, refuse while publishing, run the entity change, unschedule if needed, save.
- `exclusiveAI()`: a 180 s cache key so only one AI rewrite runs per item. A double click gets 409 `CONTENT_BUSY` instead of two paid calls. The key is always released in `finally`.
- Rewrite and regenerate check `assertEditable` **before** spending money. Delete refuses while publishing; versions cascade. Restore copies an old version forward as a new version.

application/use-cases/calendar.use-cases.ts, recover-stuck-generations.use-case.ts

- List rejects `to < from` and ranges over 400 days. Schedule is idempotent per item (move or create). Unschedule re-reads inside the lock, so two deletes at once are harmless.
- Recovery fails jobs with no progress for 30 minutes; `checkpoint(guard)` stops if the scheduler lock is lost.

infrastructure/persistence/prisma-content.repositories.ts

- Four adapters, every `where` includes `churchId`. `save()` upserts the item then `createMany` its new versions; the unique `(content_item_id, version)` index makes a concurrent writer fail instead of overwriting history.
- Version lists are capped at 100 and calendar ranges at 500. `list()` hides ARCHIVED by default and runs count + page in parallel.

presentation/

- `content.controller.ts`: `PatchBody` is `.strict()`. Rewrite and regenerate have `@Throttle(AI_THROTTLE)`.
- `generation.controller.ts`: returns **202**; 1 to 8 known content types.
- `calendar.controller.ts`: times are UTC; the web app converts from Lagos time.
- `content.responses.ts`: explicit DTOs, so a new DB column never leaks.
- `content-job.handlers.ts`: registers the `generate` handler and the 10-minute recovery job.
- Tests cover versioning, DRAFT reset on edit, scheduling rules, `planGeneration`, canon-only scripture and verbatim quotes.

7.5 Brand Studio (src/modules/brand)

The church saves its look (colours, fonts, logo, handle) and voice. Any content item can then be rendered into a branded image, a carousel, or a captioned vertical clip cut from the sermon video.

Save brand: `PUT /brand-studio` → `UpdateBrandProfileUseCase` (check logo uploads in storage first, then lock, `applyBrandPatch`, save). **Logo upload:** presign → PENDING asset row → browser uploads directly → `PUT /brand-studio` with `logoAssetId` confirms it.

Render: `POST /content/:contentId/render` → `RequestContentRenderUseCase` (pick template, snapshot brand, plan slides or clip + captions, create PENDING `MediaAsset` rows with an exact `renderSpec`, enqueue) → 202 → worker: `RenderMediaGroupUseCase` → `MediaRenderer` (sharp for images, ffmpeg for video) → `putObject` → `markReady`. Every 15 minutes, `recover-stuck-renders` fails renders pending over an hour and deletes logo uploads never attached within a day.

brand.module.ts, ports

- Binds `SERMON_MEDIA` to `SermonMediaAdapter` and exports `BrandProfileService` (Reels uses it).
- `brand.ports.ts`: profile repository (one per church), media asset repository, `BrandRenderingPort`, `MediaSettings`, `MediaObserverPort`. `markReady` only moves PENDING → READY, which makes double runs safe. The renderer writes only inside a caller-owned scratch folder and accepts `signal`.
- `sermon-media.port.ts` + `sermon-media.adapter.ts`: read access to a sermon's original video and word timings. `removedAt` says the retention sweep deleted the original.

media-presenter.ts, media-metrics.ts

- A download URL is created only for READY assets, signed and valid for 15 minutes. `media_render_duration_seconds` is a Prometheus histogram by kind and outcome.

domain/brand-profile.ts

- `BrandProfilePatch`: `null` clears a field, `undefined` leaves it alone.
- `FONT_NAME` is a deliberately strict regex because font names are written into SVG and caption files (stops injection). Colours must be `#RRGGBB`.
- `applyBrandPatch()` checks every rule, normalises `@handle` and `#hashtags`, and returns a new object.
- `BrandSnapshot` is a frozen copy stored with each render, so later brand edits don't change a render already queued.

domain/video-settings.ts, templates.ts

- `normalizeVideoSettings()` reads stored JSON **leniently** (unknown values fall back to defaults), so adding a setting never breaks old rows. `mergeVideoSettings()` applies updates **strictly**.
- `ASPECTS` (square, portrait, story), `VIDEO_FRAME` 1080×1920, 4 `TEMPLATES` as plain data (drawn by code, never by a generative model). `resolveTemplate()` picks requested → church default → catalog default.

domain/template-svg.ts, carousel-planner.ts, clip-window.ts, captions.ts

- `escapeXml()` escapes all user text, so no markup can be injected. `readableOn()` picks black or white text for contrast. `fitText()` finds the biggest font that fits. The SVG has no external references; the logo is composited later. The same input always gives the same image.
- `planCarousel()`: cover, ~280-character body slides, optional call-to-action, max 10 slides.
- `resolveClipWindow()`: the user's window (5 s minimum, configured maximum, inside the sermon), or the first minute of the source passage.
- `buildCaptionCues()`: short cues (max 6 words / 34 characters), break at sentence ends, no overlaps.

domain/render-spec.ts, media-asset.ts

- `RenderSpec` is exactly what the worker must draw, stored on the PENDING asset with no secrets. `validateLogoUpload()`: PNG/JPEG/WebP only (no SVG), max 5 MB. Keys go under `churches/{churchId}/…`.

application/use-cases/brand-profile.use-cases.ts

- `BrandProfileService`: load saved or default profile, build snapshots, return signed logo URLs.
- `UpdateBrandProfileUseCase`: storage checks **before** the lock (no network calls while holding a lock), then lock, patch and save.
- Presign (15-minute upload URL), delete (idempotent; storage object deleted last), templates, and `PreviewBrandUseCase`, which returns the same SVG the worker would draw, with unsaved changes applied.

application/use-cases/render.use-cases.ts

- `RequestContentRenderUseCase`: refuses archived content, builds PENDING assets with fixed object keys, saves in a transaction and enqueues with job id `render:{renderGroupId}` so the same render can't be queued twice. Video needs a source

video that hasn't expired.

- `RenderMediaGroupUseCase` (worker): renders only PENDING assets, so a rerun is safe. On retry it reuses the stored clip for the thumbnail instead of re-encoding. If an asset was deleted mid-render, the uploaded object is deleted so nothing is orphaned. Storage bytes are metered with a dedupe key.

infrastructure/

- `prisma-brand.repositories.ts`: one profile per church (upsert by `churchId`). `markReady` / `markFailed` use `updateMany … status: 'PENDING'` as compare-and-set.
- `rendering/media-renderer.ts`: `sharp.concurrency(1)` and `cache(false)` limit memory. Images are JPEG with a 40M-pixel input limit on logos. Video runs ffmpeg with a 20-minute timeout and the abort signal; ffmpeg failures are permanent (no pointless retries).
- `rendering/ffmpeg-clip-args.ts`: arguments as an array (no shell injection). Cut, blurred 9:16 background, burnt-in captions, logo overlay, H.264/AAC MP4 with `+faststart`. `-protocol_whitelist file` and `-nostdin`.
- `rendering/ass-captions.ts`: writes the ASS subtitle file; `assText()` escapes `{ }` and `\` so captions can't inject override codes.

presentation/

- `brand-studio.controller.ts`: every body uses `z.strictObject` with max lengths.
- `content-media.controller.ts`: render is AI-throttled and returns 202; only video may have a clip window.
- `brand-job.handlers.ts`: one render handler on two queues (image and video, so slow videos don't block images), plus the 15-minute recovery job.
- Tests cover profile rules, SVG escaping, text fitting, carousels, captions, clip windows, ffmpeg arguments and a real sharp render.

7.6 Reels (src/modules/reels)

From a sermon video, the AI suggests 5-10 standalone moments ("candidates"). The church picks one or makes its own, adds an optional on-screen hook and a caption style, and the worker renders a branded 9:16 vertical video with ffmpeg. The pastor's real video and voice are always used, never replaced.

How it flows:

- **Candidates (automatic):** the sermon pipeline finishes and enqueues `GENERATE_REEL_CANDIDATES_JOB` → `GenerateReelCandidatesUseCase` → transcript → `toSentences` → `ReelCandidateAIService.propose` (LLM) → `selectReelCandidates` (domain checks) → `replaceForSermon`. Manual regeneration enqueues the same job with a `requestId`.
- **Create and edit:** `POST /reels` (from a candidate or a custom window) → `PATCH /reels/:id` → `applyReelEdit`. **Hooks:** `POST /reels/:id/hooks` → AI → `cleanHooks`.
- **Render:** `POST /reels/:id/render` → `RequestReelRenderUseCase` (plan quota under a lock, QUEUED, `renderAttempt+1`) → `reel-render` job → download source → probe → `SpeakerTracker` + `planCropPath` → `buildReelAss` + `buildReelArgs` → ffmpeg → store → `completeRender` → usage metered.
- Every 15 min `recover-stuck-reels` fails renders with no progress for 1 hour.

reels.module.ts

- Imports `BrandModule`. Binds the **bridge** ports `REEL_SOURCE`, `REEL_BRAND` and `REEL_MEDIA` to adapters that call other modules; repositories, renderer and tracker are bound in composition.

domain/reel.ts

- Render stages, statuses and `STAGE_PROGRESS` (the percentage the UI shows).
- `renderAttempt` goes up with every render request, and jobs carry it, so a stale job knows to stop. `editedSinceRender` means the stored video is out of date.
- `validateWindow`: whole milliseconds, within min/max length, not past the end. Hooks are max 90 characters with wrapping quote marks removed, so a hook never looks like a quote from the pastor.
- `applyReelEdit` refuses edits while rendering (409). A **visual** change on a rendered Reel sets `editedSinceRender` and clears approval; a title-only change does not.
- `needsRender` is false while rendering or when completed and unchanged, which makes the render request idempotent. Approval needs the latest completed render.

domain/reel-candidates.ts, transcript-sentences.ts

- The AI proposes **sentence indexes**, not timestamps. Defaults: 15-90 s, at most 10 candidates, at most 30% overlap. `weightedScore` clamps scores to 1-10.
- `selectReelCandidates` snaps to whole sentences, grows or shrinks to fit, drops anything invalid and builds the transcript from the real sentences, **never from the model's text**. Sorted best first, overlaps skipped.
- `toSentences`: a sentence ends at `.!?`, after a 1.2 s+ pause, or after 45 words (preachers run sentences together).

domain/crop-path.ts

- Pure maths that turns landscape video into a 9:16 crop that follows the speaker. A 250 ms grid, samples under 50% confidence ignored, a 12% dead zone (no jitter on gestures), pans of at most 0.35 crop-widths per second, at most 120 keyframes.
- Holds the nearest sample, median-filters 5 points to remove spikes, applies the dead zone and speed limit, then simplifies with Ramer-Douglas-Peucker so the ffmpeg expression stays small. A narrow source returns `null` (fit, don't crop).

application/ports/out/reel.ports.ts, reel-jobs.ts, reel-presenter.ts

- `advanceRender`, `completeRender` and `failRender` take `renderAttempt` and return `false` when the Reel is no longer on that attempt. This is a **fencing token** that stops stale workers.
- Renderer and tracker ports work on local files and accept an `AbortSignal`. Media asset ids are derived from (Reel, attempt), so a retry overwrites the same objects.
- Render job id is `reel-render:<id>:<attempt>`. The presenter never includes storage keys; list views skip signed URLs (one storage call per row would be wasteful), detail views add 1-hour signed URLs.

application/services/

- `reel-candidate-ai.service.ts`: Zod schema (max 15 items); score bounds are enforced by the domain because providers' strict-JSON modes differ. Temperature 0.3, sentence numbers only, no made-up content, `signal` passed down.
- `reel-hook-ai.service.ts`: temperature 0.7 with the church's brand voice (tone, emoji rule, phrases to avoid). `cleanHooks` enforces in code: no quote marks, max 90 characters, no duplicates or avoided phrases, at most 3.

application/use-cases/

- `reel-candidates.use-cases.ts`: only READY sermons with `sourceType === 'MEDIA'`. Skips jobs from older processing runs. An automatic run does nothing if candidates exist; only a manual run replaces them, inside one transaction. Regeneration checks the AI budget.
- `reels.use-cases.ts`: a per-Reel lock serialises edits, hooks, approval and delete. Hooks need the `reelHooks` plan entitlement (402 otherwise). Delete keeps rendered assets because a scheduled post may still use them.
- `reel-render.use-cases.ts`: output 1080×1920. `RequestReelRenderUseCase` returns early when nothing needs rendering (no double charge). Under the lock `reel:quota:<churchId>` it counts used + in-flight renders against the plan, so two requests can never both take the last slot. If enqueueing fails, it marks FAILED. `RenderReelUseCase`

skips stale attempts, works in a scratch folder that is always cleaned up, `advance(stage)` throws Superseded if the attempt moved on, discards output if `completeRender` returns false, and meters with dedupe keys.

infrastructure/

- `reel-source.adapter.ts`, `reel-brand.adapter.ts`: one-way bridges to the sermons and brand modules.
- `reel-media.adapter.ts`: `deterministicId("reel:<id>:<attempt>:video")` so a retry overwrites the same object; inserts only missing asset rows; signed URLs valid for 1 hour.
- `reel-metrics.ts`: render duration histogram (10 s to 25 min), outcome counter and candidate counter.
- `persistence/prisma-reel.repositories.ts`: every query uses `{id, churchId}`. Render updates use `WHERE renderAttempt = ? AND status IN (rendering stages)`, so the database decides who is current. Queries in a transaction run one after another because it has a single connection.
- `tracking/command-speaker-tracker.ts`: runs the Python tracker via `runProcess` (no shell), validates its JSON with Zod (max 20,000 samples) and falls back to centre crop on any failure. `center-speaker-tracker.ts` is the default.
- `rendering/ffmpeg-reel-renderer.ts`: `parseProbe` swaps width and height for rotated phone videos. ffprobe has a 60 s timeout, the encode 20 minutes. The thumbnail is grabbed 60% through the hook so the hook text appears on the cover.
- `rendering/reel-ffmpeg-args.ts`: the moving crop as a sum of gated linear pieces (not nested, so ffmpeg's parser copes). Audio clean-up (high-pass, light denoise, compressor, loudness normalised to -14 LUFS) only cleans the real voice. Then the outro, burnt-in subtitles and logo. H.264 `veryfast` CRF 21, AAC 160k, 30 fps, `+faststart`.
- `rendering/reel-ass.ts`: the subtitle script libass burns in. Text colour is chosen by WCAG luminance; captions stay out of TikTok/Reels button zones. Five hook animations (IMPACT, QUESTION, MINIMAL, HIGHLIGHT, TYPEWRITER using `Intl.Segmenter` so emoji are never split) and four caption styles. All text goes through `assText` escaping.

presentation/

- `reels.controller.ts`: times are whole milliseconds up to 24 h; `z.strictObject` everywhere. Candidate regeneration and hooks are AI-throttled; render returns 202.
- `reel.responses.ts`: explicit DTOs. `reel-job.handlers.ts`: candidate, render and recovery handlers.

- Tests cover sentences, candidate selection, crop paths, the Reel lifecycle, ASS escaping, ffmpeg arguments, rotation, the no-shell tracker, church scoping and a real end-to-end ffmpeg render.

7.7 Integrations (src/modules/integrations)

A church connects its outside accounts once: a Facebook Page (META), WhatsApp Business, Mailchimp (EMAIL) and YouTube. The backend runs the OAuth sign-in flow and stores the tokens encrypted.

How it flows: `POST /integrations/meta/connect` → `ConnectIntegrationUseCase` stores a random `state` in the cache and returns the provider's authorization URL. Facebook redirects to `GET /integrations/meta/callback?code&state` → `CompleteOAuthUseCase` takes the state out of the cache, checks the session's church, exchanges the code for tokens, encrypts them with `SecretBox.seal`, upserts the row and redirects back to the web app.

integrations.module.ts, domain/integration.ts

- `@Global()`, exporting only `IntegrationCredentialsService`, so publishing can inject it without import cycles.
- Four providers. One row per church per provider; `encryptedCredentials` is never sent to the browser.

application/ports/out/integration.ports.ts

- `IntegrationRepositoryPort` (every method takes `churchId`). `OAuthProviderPort`: a `configured` flag, `authorizationUrl()` and `exchange()`.

application/use-cases/integration.use-cases.ts

- The OAuth state lives in the cache for 10 minutes. `ConnectIntegrationUseCase` refuses providers whose keys are missing and generates an unguessable single-use `state` tied to `{churchId, provider}`. This is the OAuth CSRF protection.
- `CompleteOAuthUseCase` uses `cache.take` (read **and delete**, so single use) and requires the signed-in church to be the one that started the flow. That stops an attacker sending their link to a victim and capturing the victim's Facebook Page. It never throws to the browser; it always redirects with `?connected=` or `?error=`. Reconnecting updates the existing row.
- Disconnect is idempotent. `IntegrationCredentialsService` decrypts only for `CONNECTED` rows; `markNeedsAttention` is called by publishing on a 401/403.

infrastructure/oauth/oauth-providers.ts

- `getJson` uses `fetch` with a 15 s timeout, and its error message includes only the host, so tokens in query strings are never logged.
- Meta: short-lived token → long-lived token → first Page's token. WhatsApp: token stored for later (sharing is manual in V1). Mailchimp: token → account metadata → first audience. YouTube: `access_type=offline` and `prompt=consent` make Google return a refresh token.

infrastructure/persistence/prisma-integration.repository.ts, presentation/http/integrations.controller.ts

- An explicit column list (new columns never leak). The `churchId_provider` unique key makes upsert idempotent.
- The callback uses `@OptionalAuth()` so it loads without a session, and the use case then rejects a missing or mismatched session. The cookie arrives because it's a top-level redirect with `SameSite=Lax`. `CallbackQuery` caps the length of `code`, `state` and `error`.

7.8 Publishing (src/modules/publishing)

When a calendar item's time comes, a background worker posts it to Facebook or sends it as a Mailchimp campaign, **exactly once**, even through crashes and retries. There is no controller.

How it flows: every 60 s `dispatch-due` → `DispatchDuePublicationsUseCase` → `releaseStale` + `claimDue` (SQL `SKIP LOCKED`) → one `publish` job per due item → `PublishCalendarItemUseCase` → pick channel → decrypt credentials → `publish_attempts` row with a unique idempotency key → Meta or Mailchimp publisher → record outcome → mark PUBLISHED or FAILED.

domain/channels.ts

- `CHANNEL_FOR_TYPE` maps each content type to a channel; `null` means "share by hand" (WhatsApp, video ideas).

application/ports/out/publishing.ports.ts

- `PublishResult` is the key idea: on failure it says whether it is `retryable`, whether it is `definite` (the provider certainly did **not** post) and whether `credentialsInvalid` is set.
- `PublishingRepositoryPort`: `claimDue` (atomic, SKIP LOCKED), `releaseStale`, `createAttempt` (false on a duplicate key), `reopenAttempt` (definite failures only).

application/use-cases/publishing.use-cases.ts

- Batches of 100; a claim older than 30 minutes is stale. Dispatch calls `checkpoint(guard)` and gives each job a unique id per claim time; "exactly once" is enforced by `publish_attempts`, not by BullMQ.
- A defensive re-read: if the entry is no longer `PUBLISHING`, or was rescheduled, the job quietly stops.
- Credentials and adapter are checked **before** any attempt row exists.
- **Idempotency** with key `calendarItemId:scheduledAt`: SUCCEEDED → just mark success. PENDING → a previous run crashed mid-call, so mark it failed and **never re-post** (a duplicate public post is worse than a visible failure). FAILED → reopen only if the failure was definite. None → create one; `false` means another worker has it.
- After a successful post the database writes are retried 3 times, because the post is already live. On failure: record the outcome, mark the integration NEEDS_ATTENTION on 401/403, and rethrow **only** for a definite, retryable failure (429) that isn't the last attempt.

infrastructure/providers/publishing-providers.ts

- `failure()`: a 4xx is definite; 429 is the only retryable code; 401/403 means invalid credentials; a 5xx is **not** definite (the provider may have posted before failing). Network errors and timeouts are "unknown".
- `call` uses `fetch` with a 30 s timeout and returns errors as values. Meta posts to `/{pageId}/feed`. Mailchimp creates a campaign titled `abyde:<idempotencyKey>`, sets content and sends; after a definite rejection it deletes the draft. Plain text is HTML-escaped; user HTML is never passed through.

infrastructure/persistence/prisma-publishing.repository.ts, presentation/queue/

- `claimDue`: one SQL `UPDATE … WHERE id IN (SELECT … FOR UPDATE OF c SKIP LOCKED)`, so two workers can never claim the same row. `releaseStale` resets stuck rows only when no attempt exists. `createAttempt` turns a unique violation into `false`.
- `publishing-job.handlers.ts` registers `dispatch-due` (every 60 s) and `publish`.
- Tests prove: publishes once, never retries an unknown outcome, retries definite retryable failures, and never re-posts after a crash that left a PENDING attempt.

7.9 Social (src/modules/social)

The biggest module (60 files, ~13,500 lines). A church connects its own accounts (Facebook Page, Instagram professional account, TikTok, YouTube channel, X, LinkedIn profile or company Page), writes one post, picks which accounts get it, and publishes now

or later. It shows status per platform ("published on Facebook, failed on X"), lets the user retry failed platforms and later collects likes, comments and views.

Key words: a **connection** is one connected account (tokens encrypted). A **post** is what the user wrote; a **publication** is one post going to one account, so a post with 3 targets has 3 publications that succeed or fail independently. The **decisive call** is the one HTTP call that makes something public. If that call times out, we don't know whether it posted, so it is **never retried automatically**: a duplicate public post is worse than a visible failure.

Flow 1, connecting (OAuth): `GET /social/connections/:provider/authorize` → `StartSocialAuthorizationUseCase` (state + PKCE verifier stored in the cache for 10 min) → provider consent page → `GET /social/oauth/:provider/callback` (`@OptionalAuth`) → `CompleteSocialAuthorizationUseCase` (takes state once, checks same church **and** user) → `exchangeAuthorizationCode` → one account: encrypt and save; several accounts: stash sealed tokens as a "selection" for the user to pick → always a 302 back to the web app.

Flow 2, publishing: `POST /social/posts` → `SocialPostPlanner.plan` (validation) → save. `POST /social/posts/:id/publish` (Idempotency-Key required) → `PrePublishCheck` → under the post lock flip each publication to `queued` → one `social-publish` job per publication. Worker: `prepare()` (load, decrypt or refresh the token, build media URLs) → **claim** queued → publishing (only one worker wins) → `adapter.publish` → `apply()` (published: save id, meter, audit; processing: enqueue a delayed `check` job) or `failed()` → `recompute` the post status.

Flow 3, recurring jobs: every minute `DispatchDueSocialPostsUseCase` (scheduled → queued, rescue stuck rows); every 30 min `RefreshExpiringConnectionsUseCase` ; every 6 h `CollectPublicationMetricsUseCase` .

social.module.ts

- Registers controllers and use cases. Binds `SOCIAL_MEDIA_LIBRARY` here (an internal adapter onto Brand); repositories, registry, cipher, settings and observer are bound in `src/composition/` . No request-scoped providers: `churchId` is passed as an argument.

domain/

- `social-platform.ts` : six platforms. `PROVIDER_FOR_PLATFORM` maps facebook and instagram to `meta` , which is why adapters are looked up by provider.
- `social-connection.ts` : tokens are stored **encrypted** only. `AuthorizedAccount` (decrypted, in memory) is never persisted, logged or returned. `needsRefresh()` is true within 5 minutes of expiry, so a publish never starts with a nearly dead token.

- `social-post.ts`: `derivePostStatus()` computes the post status from its publications; some published + some failed = `partially_published`, so one failure never hides another platform's success. You can edit only before anything was sent and can't delete while in flight or once live. `retryablePublications()` requires `confirmNotPublished: true` if any failure has `outcomeUnknown` (the duplicate-post safety net). Schedule must be 1 minute to ~1 year ahead.
- `social-errors.ts`: normalised codes (AUTH_EXPIRED, PERMISSION_DENIED, RATE_LIMITED, INVALID_MEDIA, UNSUPPORTED_CONTENT, PROVIDER_PROCESSING, PROVIDER_UNAVAILABLE, UNKNOWN). `SocialProviderError` **forces** `retryable = false` **whenever the outcome is unknown**; this single line is the core rule. `SAFE_MESSAGES` are what users see; raw messages are log-only.
- `capabilities.ts`: what each platform accepts (media types, sizes, durations, caption limits). `capabilityViolations()` returns human-readable problems and skips unknown size or duration, so a still-rendering video isn't rejected early.
- `target-settings.ts`: a per-platform whitelist. TikTok needs an explicit privacy choice, YouTube an explicit made-for-kids answer (platform policies). Unknown keys are dropped.

application/ports/out/

- `social-publishing.port.ts`: the contract every adapter implements (auth URL, code exchange, refresh, revoke, capabilities, publish, optional check, delete and metrics). Media comes as a short-lived URL **and** a storage key (for providers that need bytes). `PublishOutcome` is `published` or `processing` (polled later, never awaited in one request).
- `social-repositories.port.ts`: `transitionPublication` is **compare-and-set**: it updates only if the current status is one of `from` and returns whether it won, so exactly one worker wins each transition. `claimDue()` uses `SKIP LOCKED`. The audit log must never contain tokens.
- `social-support.ports.ts`: `TokenCipherPort` binds a `context` string into the ciphertext, so a token copied to another row or church can't be decrypted.

application/connection-access.service.ts

- The **only** place tokens are decrypted. Context is `social:<churchId>:<connectionId>:<access|refresh>`.
- `refresh()` takes a 60 s cache lock, because TikTok and X **rotate** refresh tokens and two concurrent refreshes would lock the account out. If the lock is held it throws a retryable error. It re-reads the row under the lock in case another worker already refreshed. AUTH_EXPIRED marks the connection expired.

application/post-validator.ts, social-presenter.ts

- Only IMAGE, CAROUSEL_SLIDE and VIDEO assets can be posted. Targets must be unique, in this church (another church's connection looks the same as a missing one), on the right platform and `connected`. Capability problems come back as one 422 keyed `targets.<platform>`.
- The presenter never includes tokens or raw metadata; it adds `needsReconnect` and `requiresReauth`.

application/use-cases/connection.use-cases.ts

- Start: refuses unconfigured providers, generates a state (only its **hash** is stored) and a PKCE verifier.
- `ConnectionWriter.persist()`: under a per-church lock, upserts one connection per account (reconnect updates in place), seals tokens, audits.
- Complete: never throws to the browser. `cache.take()` makes state single-use; the session must be the same church and user (CSRF and replay protection). Reconnect must return the same account or it is `account_mismatch`.
- Selection `confirm()` uses `cache.take()`, so two concurrent confirms can't both get the tokens.
- Disconnect is idempotent: under the lock it wipes tokens and sets `revoked` **first**, then revokes at the provider (best effort, only if no other live connection shares the same grant).

application/use-cases/post.use-cases.ts

- `SocialPostPlanner.plan()`: checks `contentId` belongs to the church, loads media and connections, validates, and builds one `pending` publication per target.
- Update validates outside the lock (slow I/O), then re-reads and re-checks **inside** the lock (double-check against races). Delete is idempotent and never deletes anything on the platform.
- `PrePublishCheck` re-validates right before queueing because media and capabilities can change. Publish flips publications with compare-and-set and returns 202. Schedule stores UTC plus the user's time zone (display only). Retry moves only `failed` publications back to `queued`.

application/use-cases/publishing.use-cases.ts

- Thresholds: 30 min for stuck queued/publishing, 15 min for processing, max 120 status polls, poll delay up to 300 s.

- `PublicationDispatcher.enqueue()` gives each enqueue a unique job id (BullMQ silently drops a duplicate id); exactly-once relies on the DB claim. `recompute()` runs under the post lock.
- `publish()`: no-op unless still `queued`. `prepare()` runs first, before anything can be published. Then the claim, and only the winner calls the provider. Any error after the claim is treated as "the decisive call may have happened".
- `apply()`: on published, DB writes are retried 3 times (losing "it is live" would be worse); usage is recorded once via `dedupeKey`. On processing, it gives up after 120 polls, otherwise queues a check with a growing delay.
- `failed()`: retryable and not the last attempt → put the row back and **rethrow** so BullMQ retries (the queue owns retries). Otherwise mark failed (+ `outcomeUnknown`), expire the connection on AUTH_EXPIRED, audit.
- Dispatcher (every minute): claims due rows, re-queues stuck ones, and marks `publishing` rows stuck 30+ minutes as failed + unknown (a worker died mid-call; never auto-retried). Refresh (every 30 min) handles tokens expiring within 2 h. Metrics (every 6 h) always saves `metricsUpdatedAt` so one bad post can't block the batch.

presentation/

- `social-connections.controller.ts`: `z.strictObject` with bounded sizes. Authorize and reconnect use `CREDENTIAL_THROTTLE`. The callback is the only `@OptionalAuth()` route and always answers 302.
- `social-posts.controller.ts`: caption up to 63,206 characters (Facebook's max), up to 35 media and 20 targets. Publish and retry carry `@RequireIdempotencyKey()` and return 202.
- `social-job.handlers.ts`: registers publish, status-check and the three recurring jobs, passing the job context (attempt, signal, final-attempt flag) through.

infrastructure/ (shared parts)

- `social-media-library.adapter.ts`: reads Brand Studio media (never raw sermon uploads), church-scoped, and makes time-limited delivery URLs.
- `social-metrics.ts`: Prometheus counters and histograms. Labels never include church ids, because unbounded labels would blow up Prometheus memory.
- `social-provider.registry.ts`: fails at boot if a provider is missing or an adapter claims a platform it doesn't own.
- `persistence/prisma-social.repositories.ts`: tenant queries filter by `churchId`; `transitionPublication` is `updateMany where {id, churchId, status in from}` returning `count === 1`; `claimDue` is a parameterised `UPDATE … FOR UPDATE SKIP`

`LOCKED … RETURNING`. **Watch out:** `save()` for connections upserts by `id` only, and its comment mentions a "follow-up guard below" that doesn't exist. Tenancy there relies on callers passing only ids they loaded with `churchId` (which they currently do).

- `providers/shared/social-http.client.ts`: the single HTTP path for every provider. A `DependencyGuard` per provider (opens after 5 failures for 60 s, max 16 concurrent); only PROVIDER_UNAVAILABLE counts as a failure. Timeout combined with the job signal via `AbortSignal.any`; `redirect: 'error'`. A timeout on a **decisive** request means "unknown" and not retryable. One attempt per call; the queue owns retries.
- `providers/shared/media-download.ts`: streams objects to scratch disk; `stream(start, end)` gives byte ranges so big videos are never held in memory. `pkce.ts`: the RFC 7636 S256 challenge.

Provider adapters (meta, youtube, tiktok, x, linkedin)

Each provider folder has the same files: `*.types.ts` (shapes), `*.errors.ts` (maps the provider's error codes to the normalised codes, applying the decisive/unknown rule), `*-api.client.ts` (URLs and headers, through `SocialHttpClient`), `*-oauth.service.ts`, `*-social.adapter.ts` and a spec with a stubbed `fetch`. No provider SDKs are used.

Provider	OAuth	How publishing works	Notable details
Meta (Facebook + Instagram)	Least-privilege scopes; short → long-lived token → Page tokens (never expire, so no refresh); revoke does nothing (it would disconnect every Page)	Facebook: `/photos`, `/videos`, or unpublished photos then one `/feed` post; Reels via start → Meta pulls file → finish → poll. Instagram: build a container (not decisive) → `media_publish` (decisive)	Tokens go in the `Authorization` header, never in URLs. Instagram is JPEG only, caption 2,200
YouTube (Google)	`youtube.upload` + `youtube.readonly` only; `access_type=offline`, `prompt=consent`, PKCE	Resumable upload streamed in 8 MiB chunks (5 min timeout each); only the final chunk is decisive	Session URL accepted only on `*.googleapis.com` so the token never goes elsewhere. Privacy defaults to `private`. Thumbnail is best effort

Provider	OAuth	How publishing works	Notable details
TikTok	`user.info.basic` + `video.publish`; refresh tokens rotate	TikTok **pulls** media from our URL; the init call is decisive; status is then polled	Posts are forced to SELF_ONLY until the app is audited. 64-bit ids re-parsed as strings so they aren't rounded
X (Twitter)	PKCE + Basic client auth; `offline.access`, `media.write`; refresh tokens rotate	Images one-shot (≤ 5 MB); video init → append 4 MiB segments → finalize; only `POST /2/tweets` is decisive	280 characters, 4 images, 512 MB / 140 s video
LinkedIn	Member scopes, plus organisation scopes if enabled; one account per administered Page (max 50)	Text, single image, multi-image or video (byte ranges with ETags); `createPost` is decisive	`escapeLittleText()` escapes LinkedIn's reserved characters but keeps hashtags clickable. API version header set in one place

7.10 Billing (src/modules/billing)

A church picks a plan (Free, Starter, Growth, Ministry), pays through Paystack, and gets a monthly allowance of sermons and Reels. Paystack reports payments with **webhooks** (server-to-server messages).

How it flows: checkout: `POST /billing/checkout` (Idempotency-Key required) → `CreateCheckoutUseCase` → `PaymentProviderPort.createCheckout` → hosted payment URL. Webhook: `POST /webhooks/billing/paystack` → `HandleBillingWebhookUseCase` → verify HMAC signature → parse into a `BillingEvent` → one transaction: record the event id (stops replays), find the church, per-church lock, update `subscription` and `usageAllowance`. Limit check: sermons calls `SERMON_QUOTA.assertCanCreateSermon` → `EntitlementService.usageView`.

billing.module.ts

- `@Global()`, exporting `SERMON_QUOTA` and `PLAN_ENTITLEMENTS` so other modules can check plan limits without importing billing (avoids import cycles). `useExisting` means the same object, not a copy.

domain/plans.ts

- The price list and feature table for every plan. Prices are in **kobo** (₦1 = 100 kobo), so money is always a whole number and never a rounding-prone float.
- Every plan check reads this table, never a plan id. To change what a plan gets, edit the numbers here instead of writing `if (plan === 'growth')`.

domain/subscription.ts (+ spec)

- `entitlementFor(subscription, now)` is the single rule for "what may this church use right now": ACTIVE or NON_RENEWING inside the period → the paid plan; PAST_DUE → the paid plan for a **3-day grace window**, so a failed card doesn't cut a church off at once; anything else → Free, counted per Lagos calendar month. It is pure (`now` is passed in), so it's easy to test.

application/ports/out/billing.ports.ts

- `SubscriptionRepositoryPort` with lookups by external ids, so a webhook can be matched to a church. `recordWebhookEvent` returns `false` for an event already processed (the replay guard). `BillingEvent` is provider-neutral. `PaymentProviderPort` is what any payment company must implement.

application/use-cases/billing.use-cases.ts

- `EntitlementService`: the one place plan rules are evaluated. Its three usage queries run **one after another on purpose**, because this can run inside a transaction, which has a single connection.
- `SermonQuotaService` throws `PAYMENT_REQUIRED / PLAN_LIMIT_REACHED`. It runs inside the sermon-creation lock, so two uploads can't both slip under the limit.
- `CreateCheckoutUseCase` takes the price from `PLANS` on the server, never from the client.
- `HandleBillingWebhookUseCase`, the most important code here:
  - Rejects the event unless the provider matches, the raw body exists and the **signature is valid**, so nobody can fake "payment succeeded".
  - Everything runs in one transaction. The event id is inserted first; if it exists, nothing happens (**idempotent**). If a later step fails, the insert rolls back too, so Paystack's retry is processed properly.
  - If no church matches yet (events arrive out of order), it throws `WebhookRetryLaterError`, which becomes a 503 so Paystack resends later.
  - `runLocked('billing:<churchId>')` applies one event per church at a time, because Paystack often sends `subscription.create` and `charge.success` together and both rewrite the same row.

- A plan is never granted for a charge that is too small or not in NGN. A renewal charge (within 5 days of the end) starts a new usage period.

infrastructure/

- `prisma-subscription.repository.ts`: upserts keyed by `churchId`. `recordWebhookEvent` lets the database's unique index do the dedupe, which is safe across replicas.
- `paystack.adapter.ts`: a `DependencyGuard` (8 at a time, open 30 s after 5 failures), `fetch` with a 15 s timeout, no internal retry. `verifySignature` is HMAC-SHA512 over the raw bytes compared with `timingSafeEqual`, so attackers can't guess the signature byte by byte. Paystack has no event id, so one is built from type + data id + a SHA-256 of the body. `churchId` from metadata is accepted only if it looks like a UUID.
- `fake-payment.adapter.ts`: for development and tests, signing webhooks the same way so tests can send signed events.

presentation/http/billing.controller.ts

- Checkout and portal carry `@RequireIdempotencyKey()`, so a double-click can't start two checkouts. The webhook is `@Public()` (the signature is its auth) and `@SkipThrottle()` (Paystack may send bursts), and reads `req.rawBody`.

7.11 Church (src/modules/church)

The church profile: name, colours, logo, tone of voice, Bible translation. Creating it is the last onboarding step, and the AI reads it to write in the church's voice.

How it flows: `POST /church` → `CreateChurchUseCase` → per-user lock → `Church.create` → save → `CompleteOnboardingUseCase` → return the new "me". `PATCH /church` → `UpdateChurchUseCase` → if the logo changed: verify the upload, claim + save in one transaction, delete the old logo file.

church.module.ts

- `@Global()`, exporting `CHURCH_DIRECTORY` and `CHURCH_PROFILE`, so auth (session → churchId) and AI modules can read church data without import cycles.

domain/church.entity.ts (+ spec)

- Private constructor; build with `create` or `restore`. `LIMITS` are maximum lengths that match the database and keep AI prompts small.
- `update()` trims, enforces limits, checks colours are `#RRGGBB`, turns `''` into `null`, and allows only `http(s)` websites. That last rule blocks `javascript:` links (an XSS risk). `replaceLogo()` returns the old key so the caller can delete the file.

application/

- `church-presenter.ts`: removes `userId` and the raw logo key; the logo is shown through a **signed URL that expires after 1 hour** because the bucket is private.
- `church.use-cases.ts`: `CreateChurchUseCase` uses `runLocked('church:create:<userId>')` so a double-submit can't create two churches (the unique index is the backstop); a taken slug gets a 6-character random suffix. `UpdateChurchUseCase` verifies the logo upload belongs to this church; deleting the old file is best effort. `ChurchDirectory` and `ChurchProfileService` implement the shared ports.

infrastructure/, presentation/

- The repository upserts by id and maps a unique violation to `CHURCH_EXISTS`. It looks up by `id` / `userId` because the church row *is* the tenant.
- `UpdateChurchBody` uses `.partial().strict()`. `POST /church` uses `@RequiredAuth()` instead of `@CurrentChurchId()` because the user has no church yet.

7.12 Analytics (src/modules/analytics)

The dashboard numbers. `GET /dashboard` / `GET /analytics` → use case → 30 s cache → `PrismaAnalyticsQuery`.

- `analytics.use-cases.ts`: caches per church for 30 s, so refreshes don't hit the database every time. `publishingMetrics: null`: reach and engagement are never made up. The dashboard runs three reads in parallel.
- `prisma-analytics.query.ts`: three grouped queries instead of one count per metric, all filtered by `churchId`.
- `analytics.controller.ts`: maps `latestSermon` through the sermon response mapper, so raw rows never leak.

7.13 Usage (src/modules/usage)

A log of everything a church has used (sermons processed, Reels rendered, AI cost). No controller; just a port and adapters.

- `PrismaUsageMeter.record` uses `createMany` with `skipDuplicates` on a `dedupeKey`, so a retried job is **never counted twice**.
- `PrismaAIUsageRecorder.record` writes with `prisma.client`, not `prisma.db`, so it runs **outside** any transaction and the cost trail survives even if the business operation rolls back. Cost is a `Decimal` with 6 places.
- `PrismaUsageQuery.total` is one `aggregate _sum` over the half-open range `[from, to)`.

7.14 Health (src/modules/health)

- `health.controller.ts`: `@Public()`, `@SkipThrottle()`, hidden from Swagger. **Liveness** (`/health`) never checks the database or Redis, so a database outage doesn't make the platform restart every instance. **Readiness** (`/health/ready`) returns 503 while draining or when any check fails.
- `check-readiness.use-case.ts`: runs all checks in parallel with a **2 s timeout** each, and clears its timer in `finally`.
- `shutdown-drain.ts`: on SIGTERM sets `draining = true` and waits `APP_SHUTDOWN_DRAIN_MS` before Nest closes the server. Readiness fails first, the load balancer stops sending traffic, and in-flight requests finish.

7.15 System (src/modules/system)

- `metrics.controller.ts`: `@Public()` but protected by `OBSERVABILITY_METRICS_TOKEN` compared with `timingSafeEqual`. A failed check returns **404**, not 401, so the endpoint doesn't reveal it exists. Updates queue-depth gauges before each scrape.
- `queue-inspector.ts`: waiting, active, delayed, failed and completed counts per queue.
- `purge-old-records.use-case.ts`: deletes webhook records older than 90 days and AI cost logs older than 400 days (13 months of reporting). "Older than cutoff" means a missed run just catches up.
- `system-job.handlers.ts`: the `email/send` job (so failed emails are retried by BullMQ), plus purge jobs (uploads every 6 h, old records every 24 h) scheduled through the shared registry, not `@Cron`, so only one replica runs each tick.
- `runtime-checks.ts`: reads the container memory limit from cgroup files and checks the V8 heap is 50-85% of it. In production or staging a heap that's too big **refuses to start**, so a leak ends in a heap error with a stack trace instead of a silent kill.

8. Database: Prisma, PostgreSQL and migrations

Everything a church creates lives in PostgreSQL. Four rules run through the whole schema:

- **Tenancy:** almost every table has `church_id`, and every query filters by it.
- **Ids are UUIDs made by the app** (no database default). Tables and columns are `snake_case` in Postgres and `camelCase` in TypeScript; that is what every `@map(...)` does.
- **Migrations only add things.** Removing something takes two deploys (expand, then contract) and an explicit reviewed marker.

- **Deleting a church cascades:** nearly every foreign key is `ON DELETE CASCADE`.

Changing the schema: edit `prisma/schema.prisma` → `npx prisma migrate dev --create-only` → review the new `migration.sql` (remove any lines dropping hand-written indexes) → `npm run db:migrate` → `npm run db:check`. `prisma generate` writes the typed client to `src/generated/prisma`, used only by the repository adapters. In production, migrations run as their own deploy step before the new version starts.

prisma/schema.prisma, model by model

Area	Model (table)	What it stores	Key constraints and indexes, and why
Identity	`User` (`users`)	Account; `passwordHash` only	`email` unique, stored lower-cased
Identity	`Session`	Login sessions	`tokenHash` unique (SHA-256 of the cookie, so a DB leak leaks nothing usable); `revokedAt` for logout; indexes on `userId`, `expiresAt`
Identity	`PasswordResetToken`	Reset links	Hashed token; `usedAt` makes it single-use
Church	`Church` (`churches`)	Profile, colours, AI tone settings	`userId` unique = one church per user in V1; `slug` unique
Church	`Upload` (`uploads`)	One row per presigned URL handed out	`objectKey` unique; `consumedAt` = used once; `(consumedAt, createdAt)` for cleanup
Sermons	`Sermon`	Metadata, pipeline state, AI themes/scriptures copied in for filtering	`(churchId, sermonDate desc)` for the library; GIN indexes on array columns (fast "has tag X"); `(status, updatedAt)` for the stuck sweep
Sermons	`SermonAsset`	Original uploaded file	`expiresAt` / `deletedAt` implement plan-based retention
Sermons	`Transcript`, `TranscriptSegment`	Full text; timed segments with word timings	`(transcriptId, sequence)` unique, so a retried job can't insert a segment twice

Area	Model (table)	What it stores	Key constraints and indexes, and why
Sermons	`SermonIntelligence`	AI summary as a JSON document	One per sermon; validated in the domain, not the DB
Sermons	`ScriptureReference`	Bible references with a `verified` flag	`(churchId, book)` for "sermons on Romans"
Sermons	`SermonEmbedding`	Text chunk + 1536-number vector	`Unsupported("vector(1536)")` because Prisma has no vector type; HNSW index hand-written in SQL
Content	`GenerationGroup`, `GenerationJob`	One Generate My Week click; one job per type	`(groupId, contentType)` unique: double clicks and retries can't create two jobs
Content	`ContentItem`, `ContentVersion`	Generated content and its full edit history	`(contentItemId, version)` unique stops two saves claiming the same version
Content	`CalendarItem`	Schedule entry	One per content item; `(status, scheduledAt)` for the dispatcher
Publishing	`Integration`	Connected provider per church	`(churchId, provider)` unique; `encryptedCredentials` is an AES-256-GCM blob
Publishing	`PublishAttempt`	One row per publish	`idempotencyKey` unique, so the same publish never goes out twice
Billing	`UsageEvent`	Metering rows	`dedupeKey` unique: retried jobs count once
Billing	`AIUsageLog`	Cost, latency, tokens per AI call	Nullable `churchId` and **no** foreign key, so logs survive; `Decimal(12,6)` for exact money
Billing	`BillingCustomer`, `Subscription`, `UsageAllowance`	One each per church	External payment ids unique, so a webhook maps to exactly one church
Billing	`WebhookEvent`	Processed webhooks	`(provider, eventId)` unique = replay protection

Area	Model (table)	What it stores	Key constraints and indexes, and why
Brand	`BrandProfile`	Colours, fonts, voice, video settings (JSON)	One per church
Brand	`MediaAsset`	Logos and rendered images, slides, videos	`onDelete: SetNull` from content, so deleting content keeps the image; `(status, updatedAt)` for the render sweep
Social	`SocialConnection`	Connected account, encrypted tokens	`(churchId, platform, providerAccountId)` unique; `(status, tokenExpiresAt)` for refresh
Social	`SocialPost`, `SocialPublication`	Post; one row per post per account	`(socialPostId, connectionId)` unique (idempotency anchor); `outcomeUnknown` is never auto-retried
Social	`SocialAuditLog`	Append-only history	`detail` must never contain tokens
Reels	`ReelCandidate`, `Reel`	AI-suggested windows; a reel being edited or rendered	`(sermonId, rank)` unique; `candidateId` deliberately has no FK because candidates are replaced on regeneration

Social `platform` and `status` columns are plain text checked in the domain, so a new provider state needs no enum migration.

prisma/migrations/

- `20261002210724_init`: enables the `vector` (pgvector) and `pg_trgm` (fast `ILIKE '%word%'`) extensions, then creates the original enums, 24 tables, indexes and cascading foreign keys. At the end are the **hand-written indexes** Prisma can't describe: six trigram GIN indexes (sermon title, speaker, series, main scripture; content title, body) and the HNSW cosine index on embeddings.
- `20261003090000_fk_and_sweep_indexes`: additive only. Indexes on `sermon_id` foreign keys (so cascades and joins don't scan whole tables) and `(status, updated_at)` sweep indexes. The generated `DROP INDEX` lines for hand-written indexes were

removed by hand.

- `20261003190000_brand_studio_social`: the brand and social tables.
- `20261004120000_reels_word_timings_retention`: the reel tables, new `UsageUnit` values, and nullable or defaulted columns (`deleted_at`, `expires_at`, `words`, `video_settings`), so the old version keeps working.
- `migration_lock.toml`: auto-generated; don't edit.

docs/MIGRATIONS.md and scripts/check-migrations.mjs

- The rules: expand → migrate → contract. No destructive SQL unless the file contains `-- abyde:allow-destructive` and was reviewed. Big-table indexes use `CREATE INDEX CONCURRENTLY` in their own migration. Never edit an applied migration or hand-edit production data.
- `check-migrations.mjs` fails CI on schema drift (except the allow-listed hand-written indexes) and on any destructive statement without the marker. It calls the Prisma CLI with `execFileSync` and an argument array, so there's no shell injection.

Runtime path

- Controller → use case → repository port → Prisma adapter (always `where: { churchId, … }`) → `PrismaService` (explicit pool size and `statement_timeout`) → Postgres. Inside `uow.run()`, `prisma.db` returns the current transaction, so several repository calls commit or roll back together.

9. Background jobs, queues and scheduling

Anything slow runs in the **worker** on one of 16 BullMQ queues. Every job must be safe to run twice, because a crash, a stalled lock or a shutdown kill re-runs it from the start.

How a job runs

1. A use case calls `QueuePort.enqueue(queue, jobName, payload, { jobId })`. A deterministic `jobId` (for example `sermon:<id>:<attempt>:transcribe`) means a duplicate enqueue is dropped.
2. `BullMqQueueAdapter` adds it to Redis with that queue's `attempts`, exponential backoff and retention.
3. In the worker, `BullMqWorkerHost` finds the handler in `JobHandlerRegistry` and builds an `AbortSignal` that fires on the queue's per-attempt timeout or on shutdown.
4. The handler (in a module's `presentation/queue/`) calls exactly one use case, passing the `JobContext`.

5. Errors are rethrown so BullMQ retries with backoff. `PermanentJobError` stops retrying straight away. On the final attempt, use cases mark their record FAILED with a friendly message.

Every queue and its policy

From `src/shared/infrastructure/queue/queue-policies.ts`. Backoff is the first delay, then it grows exponentially.

Queue	Attempts	First backoff	Timeout per attempt	Notes
`sermon-processing`	3	5 s	1 min	The `start` job that routes to a phase
`transcription`	4	30 s	25 min	Longest job: download, ffmpeg, chunked transcription
`sermon-analysis`	4	15 s	5 min	One AI call per sermon
`embeddings`	5	10 s	10 min	Batches of 64 segments
`content-generation`	4	15 s	5 min	One job per content type
`publishing`	5	60 s	3 min	Integrations publishing (Meta Page, Mailchimp)
`email`	6	30 s	1 min	Deleted on success (they carry single-use links)
`maintenance`	1	none	5 min	Recurring sweeps; the next tick is the retry
`brand-render`	3	10 s	3 min	Max 2 per process
`video-render`	2	60 s	25 min	Max 1 per process (ffmpeg uses every core)
`social-publish`	5	60 s	20 min	Covers a chunked video upload
`social-status`	3	30 s	2 min	Status polls re-enqueue themselves
`social-metrics`	1	none	5 min	Every 6 h

Queue	Attempts	First backoff	Timeout per attempt	Notes
`token-maintenance`	1	none	5 min	Every 30 min
`reel-candidates`	4	15 s	5 min	One AI call
`reel-render`	2	60 s	25 min	Max 1 per process

Every timeout must fit inside `WORKER_SHUTDOWN_GRACE_MS` (default 30 min), so a deploy never kills a job that could still finish.

Recurring jobs

Job	How often	What it does
`dispatch-due` (publishing)	60 s	Claims calendar items that are due and enqueues publishes
Social dispatch	60 s	Scheduled → queued; rescues stuck publications
`recover-stuck-sermons`	10 min	Fails sermons stuck 2 h so users can retry
`recover-stuck-generations`	10 min	Fails generation jobs stuck 30 min
`recover-stuck-renders`	15 min	Fails renders pending over 1 h; deletes logo uploads never attached
`recover-stuck-reels`	15 min	Fails reel renders with no progress for 1 h
Social token refresh	30 min	Refreshes tokens expiring within 2 h
`purge-expired-sermon-sources`	1 h	Deletes originals past plan retention
`purge-stale-uploads`	6 h	Deletes uploads never used within 24 h
Social metrics	6 h	Collects likes, comments, views
`purge-old-records`	24 h	Webhook records > 90 days, AI cost logs > 400 days
`purge-expired-auth`	daily	Sessions and reset tokens expired > 30 days

Why recurring jobs run exactly once

`@Cron()` would fire on every replica. Instead, each recurring job is a BullMQ job scheduler upserted by id (N replicas still make one schedule), and each tick runs inside a **Redis lock with a fencing token**. If the previous run still holds the lock, the tick is skipped rather than queued. Long jobs call `checkpoint(guard)` between batches and `assertCurrent()` before writing, so a holder that lost its lock (GC pause, network blip) can't overwrite a newer run's work. Sweeps work on **current state** ("older than X", "stuck since Y"), so a missed tick needs no replay.

The idempotency toolkit

These tricks make "safe to run twice" true across the codebase:

- **Deterministic ids:** `deterministicId(seed)` and deterministic job ids. A retry finds or overwrites what the first attempt wrote.
- **Attempt numbers as fencing tokens:** sermons (`processingAttempts`) and reels (`renderAttempt`) reject writes from stale jobs.
- **Compare-and-set writes:** `updateMany … where status = X` returning `count === 1`, so exactly one worker wins.
- **Unique constraints:** `dedupeKey` , `idempotencyKey` , `(provider, eventId)` , `(groupId, contentType)` . The database rejects the duplicate.
- **Replace instead of append:** transcripts, embeddings and reel candidates are swapped wholesale in a transaction.
- **Never repeat the decisive call:** publishing and social mark an unknown outcome as failed + `outcomeUnknown` rather than risk a duplicate public post.

10. Tests: how they work and how to write one

Tests are the safety net that lets you change the app without breaking a church's work. They prove the main journey works, that Church A can never see Church B, that payments and posts never go out twice, and that AI output is checked.

Kind	Where	What it runs	Command
Unit	`src/**/*.spec.ts`, next to the code	One function or class	`npm test`
End-to-end (e2e)	`test/*.e2e-spec.ts`	The whole real NestJS app over HTTP, with in-memory database, queue, storage, AI, email and social providers	`npm run test:e2e`

Rules: tests never touch a real database. Every port has an in-memory version in `test/support/`. Global guards, pipes and filters are `APP_*` providers, so the test app behaves exactly like production. Run `npm run verify` before finishing any change.

How an e2e test flows

- **HTTP:** `createTestApp()` → `new TestInfrastructure()` → `AppModule.register({ infrastructure: [infra.module()] })` (real app, fake infrastructure) → `configureApp()` (same as `main.ts`) → `new Client(t).post('/sermons', …)` → supertest → real guards, controllers and use cases → in-memory repositories.
- **Jobs:** `enqueue` just stores the job. The test calls `t.infra.queue.drain()`, which runs the real handlers with the real retry policy.
- **Scheduled jobs:** no cron in tests. Move time with `t.infra.clock.set(…)` and call the use case directly, for example `t.app.get(DispatchDuePublicationsUseCase).execute()`.
- **Unit tests** of adapters that call the internet replace `fetch` with `vi.stubGlobal('fetch', vi.fn(…))`, so no network is used.

test/support/: the fake infrastructure

test-app.ts

- `TEST_ENV`: dummy env values that pass Zod validation (rate limits at 10,000, silent logs, no drain delay).
- `createTestApp(env)` boots the real `AppModule` with fake infrastructure and `rawBody: true` (needed for webhook signatures). Pass extra env to test a setting, e.g. `createTestApp({ APP_RATE_LIMIT_MAX: '3' })`.
- `Client`: a "browser" for one user. It prefixes `/api/v1`, sends `Origin: http://web.test` (so CSRF accepts it) and remembers the session cookie. Helpers: `onboard(name)`, `login()`, `createSermon()` (real presign flow) and `readySermon()` (also drains the queue).

test-infrastructure.ts

- The test version of `src/composition/`: binds every port token to a fake. Handles tests use: `db`, `clock`, `queue`, `storage`, `email`, `model` (fake AI), `cache`, publishers, `paymentProvider`, renderers and one `FakeSocialAdapter` per provider.
- The fake AI is wrapped in the **real** `MeteredLanguageModel`, and search and grounded answers use the **real** adapters over fakes, so fallbacks are tested. `LocksModule` uses in-memory storage, allowed only in tests.

in-memory-db.ts, in-memory-cache.adapter.ts, in-memory-queue.adapter.ts, in-memory-storage.adapter.ts

- `InMemoryDb` is one `Map` per table. `InMemoryUnitOfWork.run()` snapshots and restores on error (real rollback), uses `AsyncLocalStorage` for nesting, and `runLocked` chains promises per key like an advisory lock. `TestClock` starts at `2026-10-05T09:00:00Z`.
- The fake cache stores JSON strings (catches values that wouldn't survive Redis), behaves like `SET NX`, `GETDEL` and fixed-window `INCR`, and is capped at 10,000 entries.
- The fake queue ignores a duplicate `jobId` (like BullMQ), uses the real `QUEUE_POLICIES` attempts and `PermanentJobError` rule, and `drain()` throws after 1,000 iterations to catch infinite job loops.
- The fake storage uses `memory://` URLs; `put()` simulates a browser upload; downloads write real temp files so the pipeline works on disk.

in-memory-repositories.ts, in-memory-social.ts, in-memory-reels.ts

- About 1,500 lines of `Map`-backed repositories. Two habits everywhere: `clone()` on every read and write (code can't mutate "stored" rows), and a `churchId` check on every method (mirrors tenancy).
- They copy the database's guarantees: unique emails, one church per user, single-use uploads and reset tokens, cascade deletes, guarded sermon updates, unique content versions, `dedupeKey` metering, webhook replay protection, compare-and-set publication transitions and reel render fencing.

fakes.ts, fake-publisher.ts, recording-email.adapter.ts, social-fakes.ts

- `FakeMediaProcessor` (every file is one 10-minute chunk), `FakeOAuthProvider`, `FakePublisher` (counts publishes to prove "exactly once"), `RecordingEmailAdapter` (tests read reset links out of emails).

- `FakeSocialAdapter`: tests script `accounts`, `publishSteps`, `checkSteps` and `refreshResult`. `FakeBrandRenderer` writes the real card SVG and can simulate an ffmpeg crash. `FakeReelRenderer` and `FakeSpeakerTracker` do the same for reels.

The e2e suites

File	What it proves
`critical-flow.e2e-spec.ts`	The happy path: register → church → upload → READY → generate week → edit → approve → schedule. `HttpOnly; SameSite=Lax` cookie, no `userId` leak, evidence links to real segments, AI logs never contain prompts
`auth.e2e-spec.ts`	Same error for wrong password and unknown email, lockout per email + IP, enumeration limiter, idempotent logout, session expiry, email change needs the password, single-use reset tokens
`billing.e2e-spec.ts`	Plan limit 402 with upload rollback, bad signature 401, wrong currency ignored, replay-safe webhooks, unknown customer → 503 and not recorded
`content.e2e-spec.ts`	Edits return to DRAFT and unschedule, history is never deleted, approve is idempotent, edited quotes lose `verbatim`
`isolation.e2e-spec.ts`	**Mandatory.** Church B's data is a 404 (never 403) for Church A across every module; a `churchId` in the body is ignored
`processing.e2e-spec.ts`	Upload validation, one upload → one sermon, retry resumes from the failed stage, partial generation failures, Ask admits no evidence
`publishing.e2e-spec.ts`	Secrets never exposed, OAuth state single-use, credentials encrypted, exactly-once publishing even on duplicate job delivery
`resilience.e2e-spec.ts`	Deletes during processing, lost jobs recovered, queue down at upload, files changed after validation, AI budgets, huge documents, search and Ask fallbacks

File	What it proves
`security.e2e-spec.ts`	CSRF, CORS allowlist, security headers, one error envelope, the full idempotency contract, 100 KB body limit, readiness during shutdown, rate limiting, metrics token
`brand-studio.e2e-spec.ts`	Strict schemas, SVG logos refused, preview escapes text, renders queued not done in-request, failed renders become FAILED
`reels.e2e-spec.ts`	Candidate rules, plan gates (402), double-click renders once, captions from word timings, delete mid-render charges nothing, source retention
`social.e2e-spec.ts`	Multi-account selection, encrypted tokens + PKCE, per-platform independent results, unknown outcome needs confirmation, token refresh, scheduling dispatches once

Things worth knowing

- A few e2e tests write straight to the fake database to reach hard-to-create states (for example `content.e2e-spec.ts` L133-134, `social.e2e-spec.ts` L562-565). That only touches `InMemoryDb`, but `CLAUDE.md` says no test writes to the database directly, so prefer reaching state through endpoints in new tests.
- `InMemoryStorageAdapter.createUploadUrl` uses `Date.now()` instead of the `TestClock`. The fake queue never aborts its signal, so cancellation paths are only covered by unit tests.
- Files using `beforeAll` share one app and run in order, so running a single `it` with `.only` may fail.

11. How to make changes: recipes

Every change follows the same direction: domain first, then the use case and port, then the adapter, then the controller or job handler, then tests. Finish every change with `npm run verify`.

Add a new endpoint

1. Write a use case class in the module's `application/use-cases/`. It takes `churchId` as its first argument, throws `DomainError` (never `HttpException`), and wraps writes in `uow.run` or `uow.runLocked(key, …)` when two requests could race.

2. Register it in the module's `providers` (`<module>.module.ts`).
3. Add a controller method: `@CurrentChurchId()` for the tenant, `@Body(zod(Schema))` with `z.strictObject` and max lengths, `ParseIdPipe` for ids. Map the result to an explicit response DTO; never return a Prisma row.
4. It is protected automatically by the global auth guard. Add `@Public()` only on purpose and call it out in review. Add `@Throttle(AI_THROTTLE)` if it calls AI, and `@RequireIdempotencyKey()` if it moves money or sends anything.
5. Write an e2e test with `createTestApp()` + `Client`, and add a case to `test/isolation.e2e-spec.ts` proving another church gets 404.

Add a field to an existing entity (example: `sermons.location`)

1. `prisma/schema.prisma`: add it as nullable or with a default, with `@map("location")`.
2. `npx prisma migrate dev --create-only --name sermon_location`, review the SQL, delete any `DROP INDEX` lines for the trigram/HNSW indexes, then `npm run db:migrate` and `npm run db:check`. Add a line to `docs/MIGRATIONS.md`.
3. Domain entity: add it to the props and validation (e.g. `updateDetails` in `sermon.entity.ts`).
4. Mapper and repository (e.g. `sermon.mapper.ts`, `updateDetails` in `prisma-sermon.repositories.ts`) and the read-model type.
5. Controller schemas and the response DTO.
6. The in-memory repository in `test/support/`, if it copies columns explicitly.

Add a new table

- A model with `id String @id @db.Uuid`, `churchId … @map("church_id") @db.Uuid`, a relation to `Church` with `onDelete: Cascade`, an index starting with `churchId` and `@@map("snake_name")`. Then a port + token, a Prisma adapter that filters every method by `churchId`, one line in `PRISMA_REPOSITORIES` in `src/composition/prisma-persistence.module.ts`, and an in-memory version registered in `test/support/test-infrastructure.ts`.

Rename or remove a column safely

- Never in one deploy. Release 1: add the new column, write both, backfill with a job. Release 2, once nothing reads the old column: drop it in a migration containing `--abyde:allow-destructive`, reviewed. Otherwise `check-migrations.mjs` fails CI.

Add a background job

1. If it needs a new queue, add the name to `QUEUES` (`src/shared/application/ports/queue.port.ts`) and a row to `QUEUE_POLICIES` (TypeScript forces this). Keep the timeout inside the shutdown grace period.
2. Register the handler with `defineJobHandler` in the module's `presentation/queue/*-job.handlers.ts`, calling exactly one use case.
3. Make it safe to run twice: a deterministic `jobId`, compare-and-set writes or unique constraints, and `PermanentJobError` when retrying can't help. Pass `context.signal` down to all I/O.
4. For a recurring job use `registerRecurring` (never `@Cron`), process current state, and call `checkpoint(guard)` between batches.

Add or swap an external provider

1. Write an adapter implementing the existing port, with an explicit timeout (`AbortSignal.timeout`), the caller's `signal`, a `DependencyGuard`, and **no internal retries** (the queue or caller owns retries).
2. Add config keys to `src/shared/infrastructure/config/env.schema.ts` (with `superRefine` rules if needed) and `.env.example`.
3. Choose it in the right factory in `src/composition/external-providers.module.ts`.
4. Add a fake to `test/support/` and a unit spec that stubs `fetch`.

Module-specific examples:

- **AI** (OpenAI → another vendor): implement `LanguageModelPort`, call `reportTokenUsage`, Zod-validate output, and wire it where `MeteredLanguageModel` is built.
- **Transcription:** implement `TranscriptionPort` (timestamps in ms relative to the chunk) in the `TRANSCRIPTION` factory.
- **Payments:** implement `PaymentProviderPort` (own signature check, stable event id, UUID-checked `churchId`); the webhook URL becomes `/webhooks/billing/<name>` automatically.
- **Social platform** (e.g. Threads): add it to `social-platform.ts` and `target-settings.ts`, create `providers/threads/` with types, errors (mark the publishing call `decisive: true`), API client via `SocialHttpClient`, OAuth service and adapter, then add it to `SOCIAL_PROVIDER_REGISTRY`. The registry refuses to boot if it's missing.
- **Publishing channel:** a `PublishingProviderPort` class classifying failures with `failure()` / `uncertain()`, added to `PUBLISHING_PROVIDERS`, and mapped in `publishing/domain/channels.ts`.

- **Reel renderer or tracker:** implement `ReelRendererPort` / `SpeakerTrackingPort` and change the factory; use cases don't change.

Common tweaks: where the knob lives

I want to change…	Edit
A plan's price, limits or features	`PLANS` in `billing/domain/plans.ts` (kobo)
What the AI writes for a content type	`CONTENT_SPECS` in `content/application/services/content-specs.ts`
Add a rewrite button	`REWRITE_ACTIONS` + `REWRITE_INSTRUCTIONS`
Add a content type	`CONTENT_TYPES`, `CONTENT_SPECS`, the Prisma enum + migration
Allowed upload file types	`POLICIES` in `uploads/domain/upload-policy.ts`
Per-church hourly AI caps	`AI_BUDGETS` in `src/shared/application/ai-budget.ts`
Per-IP rate limits	`APP_RATE_LIMIT_*`, `AUTH_RATE_LIMIT_MAX`, `APP_AI_RATE_LIMIT_MAX` env vars
Queue retries and timeouts	`QUEUE_POLICIES` in `queue-policies.ts`
Image templates	`TEMPLATES` in `brand/domain/templates.ts` (+ `renderCardSvg`)
Video or brand settings	`video-settings.ts` (stored as JSON, no migration needed)
Reel length, candidate count, camera feel	`MEDIA_REEL_*` env, `DEFAULT_CANDIDATE_RULES`, constants at the top of `crop-path.ts`
Caption look	`reels/infrastructure/rendering/reel-ass.ts`
Social polling and staleness	Constants at the top of `social/application/use-cases/publishing.use-cases.ts`; keep `STALE_PUBLISHING_MS` longer than the longest upload
Dashboard metrics	`ProductMetrics` + `prisma-analytics.query.ts`
A new environment variable	`env.schema.ts` + `.env.example` + a case in `env.schema.spec.ts`; never log the value

Before you open a PR

- `npm run verify` passes (lint, typecheck, unit + e2e tests, build).
- Every new query filters by `churchId`; every new endpoint returns a DTO.
- Every new port has an in-memory implementation in `test/support/`.
- Any migration is additive, or carries the reviewed destructive marker.
- No secrets in logs, URLs or the image.